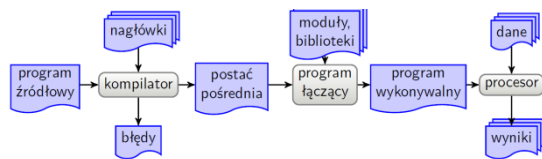
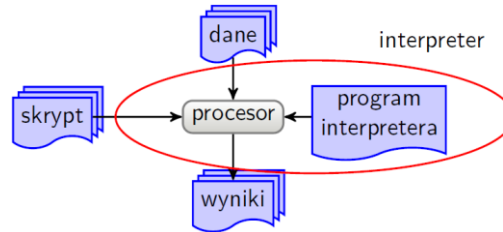


Kompilacja:



Interpretacja:

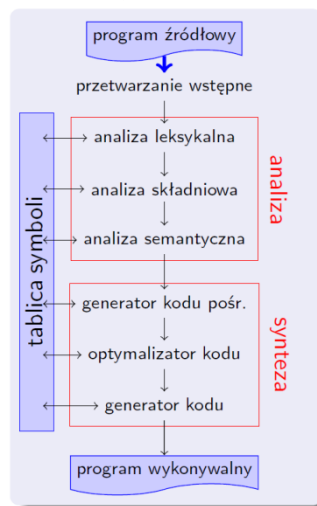


Inne schematy tłumaczenia:

- tłumaczenie na kod maszyny wirtualnej: kod źródłowy jest tłumaczony na kod pośredni; kod pośredni jest wykonywany przez maszynę wirtualną
- kompilacja dokładnie o czasie (ang. just in time): tłumaczenie jest dokonywane w trakcie wykonania programu; tłumaczony jest tylko aktualnie wykonywany fragment programu; raz przetłumaczony fragment może być wielokrotnie wykorzystywany; często tłumaczy się na najpierw na kod maszyny wirtualnej
- makroprzetwarzanie: definiowane są reguły przekształcania dopasowanego tekstu na inny

tekst; obecnie stosowane głównie jako etap przetwarzania wstępnego w kompilacji i asemblacji; obecnie bardzo rzadko stosowane samodzielnie do tłumaczenia programów

Budowa kompilatora:



Zadania przetwarzania wstępnego: makroprzetwarzanie, dołączanie plików, tłumaczenie warunkowe, ustawianie parametrów tłumaczenia.

Zadaniem analizy leksykalnej jest: rozpoznawanie elementów (tokenów/leksemów) takich jak identyfikatory, stałe liczbowe, napisy, operatory, interpunkcja itp. poszczególnym rodzajom elementów przypisane są odpowiednie numery; ustalanie wartości tych elementów, dla których ma to sens; pomijanie nieistotnych znaków; wykrywanie błędów takich jak niezamknięty komentarz, zamknięcie nieotwartego komentarza, stała tekstowa nie kończąca się w jednym wierszu.

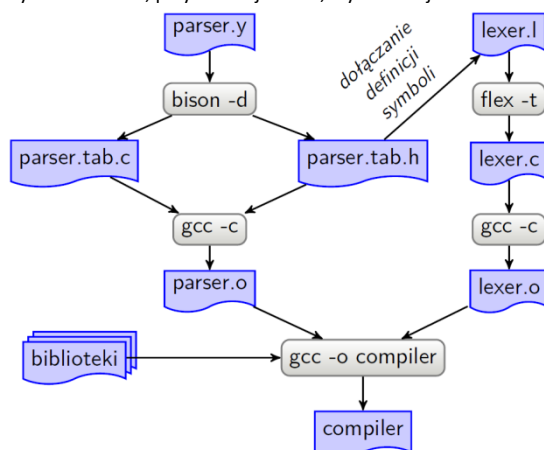
Zadaniem analizy składniowej(syntaktycznej) jest: rozpoznawanie struktury programu; utworzenie drzewa rozbioru składniowego (w sposób jawny lub niejawny); sterowanie przebiegiem kompilacji — pobieranie danych z analizatora leksykalnego, rozpoczynanie analizy semantycznej i generowania kodu dla rozpoznanych konstrukcji składniowych; sprawdzanie poprawności składniowej programu i informowanie użytkownika o błędach składniowych

Zadaniem analizy semantycznej jest: ustalanie typów stałych, zmiennych i wyrażeń; sprawdzanie zgodności typów; ustalanie przekształceń typów; ustalanie powiązań między deklaracjami i różnymi wystąpieniami stałych, zmiennych, procedur i funkcji; wykrywanie użycia zmiennych, procedur i funkcji, których nie zdefiniowano; wykrywanie użycia zmiennych, którym nie nadano wartości początkowych

Celem fazy generowania kodu pośredniego jest przekształcanie drzewa składniowego (danego w jawnej postaci lub pośrednio przez analizę sterowaną składnią) do postaci pośredniej. Postać pośrednia jest niezależna od docelowej maszyny. Jako postać pośrednia wykorzystywane są: drzewa składniowe, kod trójadresowy, zapis przyrostkowy.

Celem optymalizacji kodu jest takie jego przekształcenie, by wynikowy program wykonywał się szybciej lub używał mniejszej ilości pamięci operacyjnej. Używa się różnych technik, takich jak: usuwanie wspólnych podwyrażeń, przemieszczanie stałego kodu poza pętle, propagacja kopii, usuwanie zmiennych indukcyjnych, optymalizacja przydziału rejestrów.

Generowanie kodu jest fazą przekształcającą kod pośredni (na ogół zoptymalizowany) na kod wynikowy będący bądź to kodem maszynowym, bądź to programem w języku asemblera. W kodzie wynikowym na ogół używa się tylko adresów względnych. Umożliwia to łączenie różnych modułów programu i bibliotek. W trakcie generowania kodu rozwiązuje się takie problemy jak: zarządzanie pamięcią, wybór rozkazów, przydział rejestrów, wybór kolejności obliczeń.



FLEX

Wyrażeniem regularnym nad alfabetem Σ jest: zbiór pusty $\emptyset$; pusty ciąg symboli "" (ciąg o długości zero); symbol σ z alfabetu Σ ; sklejanie R_1R_2 dwóch wyrażen regularnych R_1 i R_2 ; alternatywa $R_1|R_2$ dwóch wyrażen regularnych R_1 i R_2 ; domknięcie przechodnie R^* wyrażenia regularnego R ; wyrażenie regularne ujęte w nawiasy; nic ponadto.

Rozszerzone wyrażenia regularne we fleksie.

. — kropka oznacza dowolny znak **oprócz znaku nowego wiersza**.

Powtórzenia. Dla wyrażenia r :

- $r\{n,m\}$ — oznacza powtórzenie wyrażenia r od n do m razy, np. $\{2,3\}$ oznacza od dwóch do trzech gwiazdek; warianty:
 - $r\{n,\}$ — r co najmniej n razy
 - $r\{,m\}$ — r co najwyżej m razy
 - $r\{n\}$ — r dokładnie n razy
- $r?$ — to samo, co $r\{0,1\}$ lub $r\{,1\}$, czyli nieobowiązkowe wystąpienie wyrażenia r
- r^* — to samo, co $r\{0,\}$, czyli dowolna liczba wystąpień wyrażenia r , włączając w to brak wystąpienia
- r^+ — to samo, co $r\{1,\}$, czyli co najmniej jedno wystąpienie r , równoważne rr^*

Kontekst. Dla wyrażen r_1 i r_2 :

- $\text{\textasciicircum}r_1$ — oznacza wystąpienie wyrażenia r_1 na początku wiersza, np. $\text{\textasciicircum}\#define$ wykrywa dyrektywę $\#define$ zapisaną na początku wiersza; uwaga: różnica pomiędzy $\text{\textasciicircum}r_1$ a $\text{\textasciicircum}nr_1$ polega na tym, że w pierwszym przypadku znak nowego wiersza nie należy do dopasowanego tekstu
- $r_1\$$ — oznacza wystąpienie wyrażenia r_1 na końcu wiersza, np. $\\. \$$ dopasowuje się do komentarza w języku C++; uwaga: różnica pomiędzy $r_1\$$ a $r_1\text{\textasciicircum}$ polega na tym, że w drugim przypadku znak nowego wiersza wchodzi w skład dopasowanego tekstu
- r_1/r_2 — oznacza wystąpienie wyrażenia r_1 , po którym bezpośrednio występuje wyrażenie r_2 , np. $[A-Za-z_][A-Za-z0-9_]* / :$ oznacza identyfikator przed dwukropkiem (prawdopodobnie etykietę); uwaga: wyrażenie r_2 nie wchodzi w skład dopasowanego tekstu

Wykonywanie reguł(flex): jeżeli do tekstu wejściowego pasuje wzorec, wykonywane jest skojarzone z nim działanie; jeżeli do tekstu pasuje więcej niż jeden wzorec, wybierany jest wzorec, który dopasowuje się do dłuższego tekstu; jeśli więcej niż jeden wzorec dopasowuje się do najdłuższego tekstu, wybierane jest działanie skojarzone z pierwszym z tych wzorców; jeżeli do tekstu wejściowego nie pasuje żaden wzorec, wykonywana jest reguła domyślna; reguła domyślna dopasowuje się do pojedynczego znaku i kopiuje go na wyjście (wypisuje go) **yymore()** — następna reguła dopisze dopasowany tekst na końcu yxtext **yywrap()** - jest wywoływana po napotkaniu końca strumienia wejściowego, powinna zwrócić 1. Nie trzeba umieszczać jej prototypu w sekcji deklaracji.

Usuwanie komentarzy w języku C

```
%x KOMENTARZ /* deklaracja warunku KOMENTARZ */
%%
<INITIAL>"/" BEGIN(KOMENTARZ); /* wejście w warunek */
<INITIAL>"/" fprintf(stderr, "Koniec komentarza bez początku\n");
<KOMENTARZ>"/" BEGIN(INITIAL); /* powrót */
<KOMENTARZ>.|\\n /* pomiń wnętrze komentarza */
%%
int yywrap(void) {
    if (YYSTATE == KOMENTARZ) {
        fprintf(stderr, "Brak zamknięcia komentarza\n");
    }
    return 1;
}
```

Rozpoznawanie napisów

```
%x NAPIS /* deklaracja warunku NAPIS */
%%
<INITIAL>" { yymore(); BEGIN(NAPIS); }
<NAPIS>" { BEGIN(INITIAL); return STRING; }
<NAPIS>. { yymore(); }
<NAPIS>\n { fprintf(stderr, "Brak końca napisu");
    BEGIN(INITIAL); }
```

```

%option yylineno

%{
#include <stdio.h> /* printf() */
#include "p.tab.h" /* deklaracja symboli końcowych */
int process_token(const char *text, const char *TokenType, const char *TokenVal, const int TokenID);
int comm_beg = 0; /* wiersz rozpoczęcia komentarza */
%}

/* deklaracja dodatkowych warunków początkowych analizatora leksykalnego */
/* (nie deklarujemy domyślnego warunku początkowego INITIAL */
%x ST_COMMENT_1 ST_COMMENT_2 ST_TEXT_CONST

/* pomocnicze wyrażenia regularne */
alpha [a-zA-Z]
/* Nazwa jest niepustym ciągiem liter, znaków podkreślenia, myślników i cyfr zaczynająca się literą lub znakiem podkreślenia.
Wzorzec jest podawany jako rozszerzone wyrażenie regularne. Deklaracja przypisuje określönemu wzorcowi podaną nazwę. Można się później do niego odwołać w części regulowej
(drugiej części) przez nazwę podaną w nawiasach klamrowych. Nazywanie wzorców nie jest konieczne. Można ich użyć bezpośrednio w części regulowej bez ich nazywania. Nazywanie
wzorców może poprawiać czytelność kodu */

%%

/* usuwanie komentarzy wielowierszowych (*.*) z wykorzystaniem mechanizmu warunków początkowych */
<INITIAL>\(\{ BEGIN ST_COMMENT_1; comm_beg = yylineno; }
<ST_COMMENT_1>\(*\) BEGIN INITIAL;
<ST_COMMENT_1>.\|\\n ;

/* usuwanie komentarzy wielowierszowych {...} z wykorzystaniem mechanizmu warunków początkowych */
<INITIAL>\{ { BEGIN ST_COMMENT_2; comm_beg = yylineno; }
<ST_COMMENT_2>\} BEGIN INITIAL;
<ST_COMMENT_2>.\|\\n ;

/* wykrycie błędu: Nieoczekiwane zamknięcie komentarza w wierszu */
<INITIAL>\)\{(\*) printf("Nieoczekiwane zamknięcie komentarza w wierszu %d\n", yylineno);

/* wykrywanie stałych tekstowych '.' z wykorzystaniem mechanizmu warunków początkowych */
<INITIAL>\' { BEGIN ST_TEXT_CONST; ymore(); }
<ST_TEXT_CONST>\' { BEGIN INITIAL; return process_token(yytext, "STRING_CONST", yytext, STRING_CONST); }
<ST_TEXT_CONST>.\|\\n ymore();

/* wykrycie dyrektyw postaci {$! name.ext} (bez wykorzystania mechanizmu warunków początkowych) */
\{$!\.+\} printf("Przetwarzanie dyrektywy INCLUDE\n");

/* wykrycie słów kluczowych bez uwzględniania wielkości liter! */
(?i:PROGRAM) return process_token(yytext, "KW_PROGRAM", "", KW_PROGRAM);

/* wykrycie symboli końcowych opisywanych złożonymi wyrażeniami regularnymi */
[A-Za-z_]{alphanum2}* return process_token(yytext, "IDENT", yytext, IDENT);
{num}+ return process_token(yytext, "INTEGER_CONST", yytext, INTEGER_CONST);
({num}*\.{num}+){({num}+\.)} return process_token(yytext, "FLOAT_CONST", yytext, FLOAT_CONST);

/* Niepotrzebne jak mamy wariant z warunkami początkowymi */
/* '\.*\' return process_token(yytext, "STRING_CONST", yytext, STRING_CONST); */

/* wycięcie białych znaków */
[ \t\n];

/* operatory wieloznakowe np.: :=, <= */
\:= return process_token(yytext, "ASSIGN", "", ASSIGN);
\<= return process_token(yytext, "LE", "", LE);

/* operatory jednoznakowe oraz interpunkcja */
[+\-*/;,:>.<>(){}] return process_token(yytext, yytext, "", *yytext);

%%

int process_token(const char *text, const char *TokenType, const char *TokenVal, const int TokenID) {
int i;
printf("%-20.20s%-15s %s\n", text, TokenType, TokenVal);
switch (TokenID) {
case INTEGER_CONST: yyval.i = atoi(text); break;
case FLOAT_CONST: yyval.d = atof(text); break;
case IDENT: strncpy(yyval.s, text, MAX_STR_LEN); break;
case STRING_CONST: l = strlen(text); strncpy(yyval.s, text + 1, l - 2 <= MAX_STR_LEN ? l - 1 : MAX_STR_LEN); break;
case CHARACTER_CONST: yyval.i = text[1]; break;
}
return(TokenID);
}/*process_token*/

int yywrap( void ){ /* funkcja wywoływana po napotkaniu końca strumienia wejściowego */
if ( YYSTATE != INITIAL ) { printf("Niezamknięty komentarz otwarty w wierszu %d\n", comm_beg); }
return( 1 ); /* koniecznie, by analiza nie rozpoczęła się od nowa */
}

```

BISON

Część regułowa zawiera ciąg reguł. Reguły składniowe mają postać:

lewa_strona: prawa_strona;

Lewa strona jest symbolem niekończącym (zmienną gramatyki). Prawa strona jest ciągiem symboli końcowych i niekończących. Ten sam symbol niekończący może wystąpić po lewej stronie więcej niż raz.

Obsługa błędów umożliwia wykrywanie błędów, sygnalizowanie ich i dalszą analizę wejścia od punktu, w którym wpływ błędu na dalszy tekst powinien być znacznie mniejszy. Odbywa się to przez standardowo zdefiniowany wirtualny (nie jest dostarczany przez analizator leksykalny) symbol końcowy error.

```
%{
#include <stdio.h> /* printf() */
void found( const char *nonterminal, const char *value );
%}
%token KW_PROGRAM KW_BEGIN KW_END KW_USES KW_VAR KW_CONST KW_IF KW_THEN KW_ELSE
%token<s> IDENT STRING_CONST
%token<d> FLOAT_CONST
%token<i> INTEGER_CONST CHARACTER_CONST

%union
{
    char s[ MAX_STR_LEN + 1 ];
    int i;
    double d;
}

/* Priorytety operatorów */
%left '+' '-'
%left '*' '/'
%right NEG

%%

/* program może być pusty (błąd semantyczny), zawierać błąd składniowy lub zawierać: nazwę programu (PROGRAM_NAME), sekcję deklaracji (SECTION_LIST) oraz blok główny (BLOCK) zakończony kropką */
Grammar: /* empty */ { yyerror( "Empty input source is not valid!" ); YYERROR; }
        | error
        | PROGRAM_NAME SECTION_LIST BLOCK '.' { found( "Complete program", "" ); };

/* Nazwa programu (PROGRAM_NAME) może nie występować lub być postaci: słowo kluczowe "PROGRAM" nazwa_programu (IDENT) średnik */
PROGRAM_NAME:
        | KW_PROGRAM IDENT ';' { found( "PROGRAM_NAME", $2 ); };

/* lista sekcji (SECTION_LIST) składa się z dowolnej (również zerowej) liczby sekcji (SECTION) */
SECTION_LIST:
        | SECTION_LIST SECTION;

/* sekcja (SECTION) może być: sekcją deklaracji stałych (CONST_SECT), sekcją deklaracji zmiennych (VAR_SECT), funkcją (FUNCTION) lub procedurą (PROCEDURE) */
SECTION: CONST_SECT | VAR_SECT | FUNCTION | PROCEDURE;

/* sekcja CONST (CONST_SECT) zawiera słowo kluczowe CONST, po którym następuje lista deklaracji (CONST_LIST) zakończona średnikiem */
CONST_SECT: KW_CONST CONST_LIST ';' { found( "CONST_SECT", "" ); };

/* lista deklaracji (CONST_LIST) zawiera jedną lub więcej deklaracji (CONST) rozdzielonych średnikiem */
CONST_LIST: CONST | CONST_LIST ';' CONST;

/* deklaracja stałej (CONST) składa się z identyfikatora, znaku równości ('=') oraz z wartości dosłownej (LITERAL) */
CONST: IDENT '=' LITERAL { found( "CONST", $1 ); };

/* DEKLARACJA FUNKCJI I PROCEDURY */

/* deklaracja procedury (PROCEDURE) składa się ze: słowa kluczowego PROCEDURE, nagłówka funkcji (FUN_HEAD), listy sekcji (SECTION_LIST) oraz bloku (BLOCK) */
PROCEDURE: KW_PROCEDURE FUN_HEAD SECTION_LIST BLOCK { found( "PROCEDURE", $<2 ); };

/* deklaracja funkcji (FUNCTION) składa się z: słowa kluczowego FUNCTION, nagłówka funkcji (FUN_HEAD), dwukropka oraz typu zwracanej przez funkcję wartości (DATA_TYPE), listy sekcji (SECTION_LIST) oraz bloku (BLOCK) */
FUNCTION: KW_FUNCTION FUN_HEAD ':' DATA_TYPE SECTION_LIST BLOCK { found( "FUNCTION", $<2 ); };

/* nagłówek funkcji (FUN_HEAD) rozpoczyna się identyfikatorem (IDENT), po którym w nawiasach okrągłych znajdują się parametry formalne (FORM_PARAMS) */
FUN_HEAD: IDENT '(' FORM_PARAMS ')' { found( "FUN_HEAD", $1 ); };

/* parametry formalne (FORM_PARAMS) mogą być puste lub mogą być listą parametrów formalnych (FORM_PARAM_LIST) */
FORM_PARAMS: | FORM_PARAM_LIST;

/* lista parametrów formalnych (FORM_PARAM_LIST) zawiera 1 lub więcej parametrów formalnych (FORM_PARAM) rozdzielonych przecinkiem */
FORM_PARAM_LIST: FORM_PARAM | FORM_PARAM_LIST ',' FORM_PARAM;

/* parametr formalny (FORM_PARAM) składa się z listy identyfikatorów (IDENT_LIST), dwukropka oraz określenia typu (DATA_TYPE) */
```

```
FORM_PARAM: IDENT_LIST ':' DATA_TYPE { found( "FORM_PARAM", "" );};
```

```
/* INSTRUKCJE PROSTE I KONSTRUKCJE ZŁOŻONE */
```

```
/* wywołanie funkcji (FUNCT_CALL) składa się z identyfikatora oraz parametrów aktualnych (ACT_PARAMS) umieszczonych w nawiasach okrągłych. Alternatywną postacią wywołania funkcji jest po prostu jej nazwa (IDENT) bez nawiasów. */
```

```
FUNCT_CALL: IDENT '(' ACT_PARAMS ')' { found( "FUNCT_CALL", $1); } | IDENT { found( "FUNCT_CALL", $1);};
```

```
/* przypisanie (ASSIGN_INSTR) składa się z identyfikatora, operatora przypisania (ASSIGN) oraz wyrażenia (EXPR) */
```

```
ASSIGN_INSTR: IDENT ASSIGN EXPR { found( "ASSIGN_INSTR", $<s>1);};
```

```
/* wyrażenie (EXPR) może być: liczbą lub identyfikatorem, sumą, różnicą, iloczynem, ilorazem oraz negacją wyrażen, a także wyrażeniem w nawiasach. Należy uwzględnić wyższy priorytet minusa jednoargumentowego!!! */
```

```
EXPR: NUMBER | IDENT | EXPR '-' EXPR | EXPR '*' EXPR | EXPR '/' EXPR | EXPR '%' EXPR | '-' EXPR %prec NEG | '(' EXPR ')';
```

```
/* instrukcja FOR (FOR_INSTR) składa się ze słowa kluczowego FOR, identyfikatora, znaku podstawienia (ASSIGN), danej CONST_VAR, słowa kluczowego TO, danej CONST_VAR, słowa kluczowego DO oraz bloku instrukcji (BLOCK_INSTR) */
```

```
FOR_INSTR: KW_FOR IDENT ASSIGN CONST_VAR KW_TO CONST_VAR KW_DO BLOCK_INSTR { found( "FOR_INSTR", "" );};
```

```
/* instrukcja if */
```

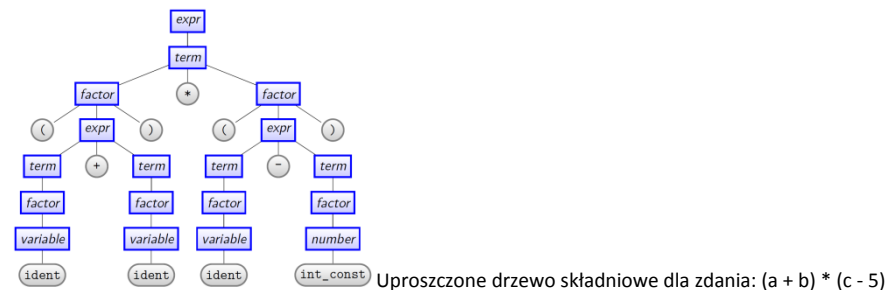
```
IF_INSTR: KW_IF EXPR '>' EXPR KW_THEN IF_INSTR_BLOCK { found( "IF_INSTR", "" );} | KW_IF '(' EXPR '>' EXPR ')' KW_THEN IF_INSTR_BLOCK { found( "IF_INSTR", "" );};
```

```
IF_INSTR_BLOCK: INSTRUCTION | INSTRUCTION KW_ELSE INSTRUCTION;
```

```
%%
```

```
int main( void ) {
    int ret;
    printf( "Autor: Imie i Nazwisko\n" );
    printf( "yytext      Typ tokena      Wartosc tokena znakowo\n\n" );
    ret = yyparse();
    return ret;
}
```

```
int yyerror( const char *txt ) {
    printf( "Syntax error %s\n", txt );
}
```



Alfabetem Σ nazywamy każdy skończony i niepusty ciąg symboli.

Zdaniem (słowem) σ nad alfabetem Σ nazywamy dowolny, skończony ciąg symboli tego alfabetu. Szczególnym przypadkiem jest słowo puste ϵ , czyli ciąg nie zawierający żadnego symbolu. Z kolei przez Σ^+ oznaczamy zbiór wszystkich niepustych słów nad alfabetem Σ . Natomiast przez Σ^* zbiór wszystkich słów nad Σ wraz ze słowem pustym ϵ .

Język L nad alfabetem Σ jest to zbiór ciągów symboli z tego alfabetu (dowolny, niepusty podzbiór Σ^*), tzn. $L \subseteq \Sigma^*$. Na przykład zbiór wszystkich nieujemnych liczb dwójkowych jest językiem nad alfabetem $\{0; 1\}$. Każdy jego podzbiór też jest takim językiem.

Gramatyką nazywamy czwórkę uporządkowaną $(V; \Sigma; P; S)$, gdzie:

- V to zbiór zmiennych (inaczej — symboli niekończących)
- Σ (oznaczany też jako T) to zbiór symboli końcowych (ang. terminals)
- P to zbiór reguł produkcji postaci $\alpha \rightarrow \beta$
- S to symbol wyróżniony — symbol początkowy gramatyki

Konkretna postać reguł produkcji zależy od typu gramatyki.

typ gramatyki	typ automatu	pamięć
G^0 – swobodna	maszyna Turinga	dostęp swobodny
G^1 – kontekstowo zależna	liniowo ograniczony	liniowo ograniczona
G^2 – bezkontekstowa	stosowy	stosowa
G^3 – regularna	skończony	brak

Maszyna turinga – stworzony przez Alana Turinga abstrakcyjny model komputera służącego do wykonywania algorytmów, składającego się z nieskończenie długiej taśmy podzielonej na pola. Taśma może być nieskończona jednostronnie lub obustronnie. Każde pole może znajdować się w jednym z N stanów. Maszyna zawsze jest ustawiona nad jednym z pól i znajduje się w jednym z M stanów. Zależnie od kombinacji stanu maszyny i pola maszyna zapisuje nową wartość w polu, zmienia stan, a następnie może przesunąć się o jedno pole w prawo lub w lewo. Taka operacja nazywana jest rozkazem. Maszyna Turinga jest sterowana listą zawierającą dowolną liczbę takich rozkazów. Liczby N i M mogą być dowolne, byle skończone. Czasem dopuszcza się też stan $M+1$, który oznacza zakończenie pracy maszyny. Lista rozkazów dla maszyny Turinga może być traktowana jako jej program.

W G^0 na reguły produkcji nakłada się jedynie ograniczenie, by po lewej stronie występował przynajmniej jeden symbol nieterminalny. Za pomocą tych gramatyk można opisać wszystkie stosowane w praktyce języki programowania. Niestety w ogólnym przypadku nie można zagwarantować, że uda się w oparciu o gramatyki tej klasy skonstruować automat analizujący zdania należące do wygenerowanego przez nie języka. Dlatego też praktyczne zastosowania gramatyk tej klasy ograniczone są głównie do prób opisu języków naturalnych.

W G^1 wymaga się dodatkowo, aby prawa strona każdej produkcji miała co najmniej tyle symboli, ile występuje po lewej stronie. Bardzo dobrze nadają się do opisu języków programowania, które zawierają liczne zależności kontekstowe. Jednak w praktyce gramatyki te nie są stosowane do opisu języków programowania ze względu na brak gwarancji istnienia automatu umożliwiającego analizę zdań kontekstowo zależnych.

W G^2 nakłada się kolejne ograniczenie - lewa strona reguły produkcji musi być zawsze pojedynczym symbolem nieterminalnym. Większość stosowanych w praktyce języków programowania opisywanych jest za pomocą gramatyk bezkontekstowych. Dowiedziono, że dla tej klasy języków można zawsze skonstruować odpowiedni analizator. Co więcej, przy spełnieniu pewnych dodatkowych założeń analizator taki można wygenerować w sposób automatyczny na podstawie reguł gramatyki.

W G^3 dodatkowym ograniczeniem jest, aby po prawej stronie reguły produkcji występował co najwyżej jeden symbol nieterminalny - gramatyki liniowe. Jeżeli wprowadzimy dalsze ograniczenia na reguły produkcji tak, by po ich prawej stronie występował zawsze jeden symbol terminalny, a ewentualny symbol nieterminalny występował zawsze po jego prawej lub lewej stronie, otrzymamy klasę gramatyk regularnych.

Gramatyką regularną nazywamy gramatykę $(V; \Sigma; P; Z)$, w której reguły produkcji mają jedną z dwóch postaci:

1. dla $A; B \in V, a \in \Sigma$

- $A \rightarrow a$
- $A \rightarrow Ba$
- $A \rightarrow \epsilon$

2. dla $A; B \in V, a \in \Sigma$

- $A \rightarrow a$
- $A \rightarrow aB$
- $A \rightarrow \epsilon$

W pierwszym przypadku mówimy o **gramatyce regularnej lewostronnej**, w drugim — o **gramatyce regularnej prawostronnej**.

Automaty skończone nazywamy skończonymi, ponieważ mają skończoną liczbę stanów. Istnieją również automaty nieskończone.

Automaty dzielimy na:

- deterministyczne (DFA)
- niedeterministyczne (NFA)
- niedeterministyczne z przejściami etykietowanymi ϵ (ϵ -NFA)

Ze względu na obecność wyjścia automaty dzielimy na:

- bez wyjścia (zwykle automaty rozpoznające język)
- z wyjściem: automaty Moore'a (wyjście w stanach), automaty Mealy'ego (wyjście na przejściach)

Automaty kompletne opisują sprzęt; wymagane są w niektórych algorytmach.

Automaty niekompletne wykorzystywane są głównie w strukturach danych w programach

Automatem skończonym deterministycznym nazywamy piątkę uporządkowaną $M = (Q; \Sigma; \delta; q_0; F)$, gdzie

- Q to skończony zbiór stanów
- Σ to skończony zbiór symboli zwany alfabetem
- $\delta : Q \times \Sigma \rightarrow Q$ to funkcja przejścia
- $q_0 \in Q$ to stan początkowy
- $F \subseteq Q$ to zbiór stanów końcowych

Przykładowy automat

$M = (\{0, 1, 2\}, \{0, 1\}, \delta, 0, \{0\})$

$\delta(0, 0) = 0, \delta(0, 1) = 1,$

$\delta(1, 0) = 2, \delta(1, 1) = 0,$

$\delta(2, 0) = 1, \delta(2, 1) = 2.$

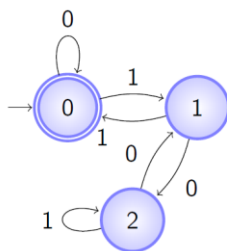


Tabela przejść

stan	wejście	
	0	1
0	0	1
1	2	0
2	1	2

Działanie

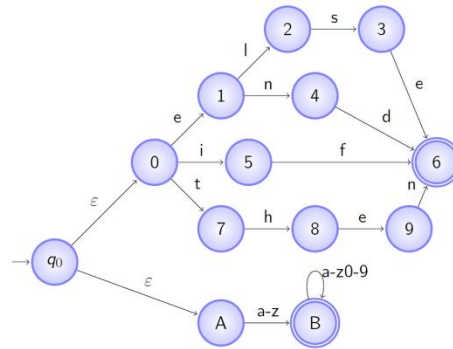
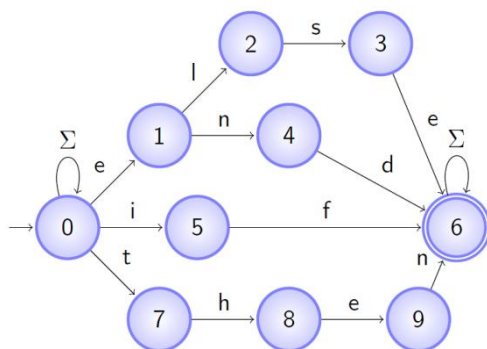
Prześledźmy działanie dla przykładowego ciągu wejściowego:

1 0 1 1 1 0 0 1 0 1
 $0 \bmod 3 = 0$

Automatem skończonym niedeterministycznym nazywamy piątkę uporządkowaną $M = (Q; \Sigma; \delta; q_0; F)$, gdzie

- Q to skończony zbiór stanów
- Σ to skończony zbiór symboli zwany alfabetem
- $\delta : Q \times \Sigma \rightarrow 2^Q$ to funkcja przejścia
- $q_0 \in Q$ to stan początkowy
- $F \subseteq Q$ to zbiór stanów końcowych

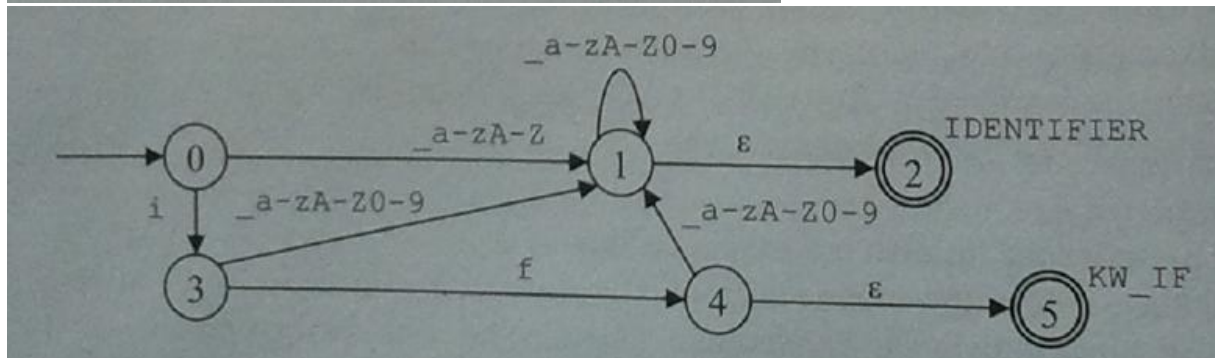
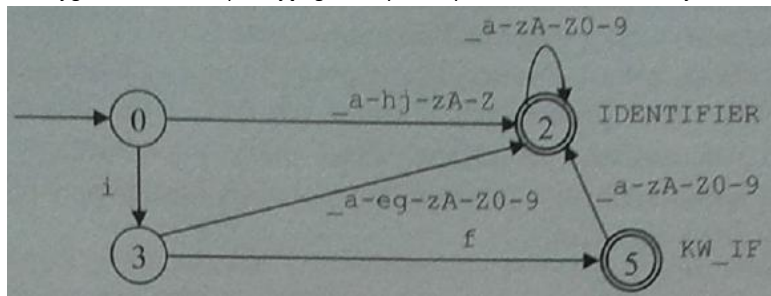
Automat rozpoznający teksty zawierające słowa kluczowe else, end, if i then w wersji NFA i ϵ -NFA



W przypadku automatów deterministycznych można jednoznacznie stwierdzić dla każdego stanu, do którego stanu nastąpi przejście pod wpływem przeczytania symbolu z wejścia lub z pamięci. Z kolei w przypadku automatu niedeterministycznego, z niektórych stanów istnieje możliwość przejścia do różnych stanów pod wpływem odczytania tych samych symboli. NFA rozważa wszystkie możliwości, po jednej na raz, aby określić, czy ciąg wejściowy może być zaakceptowany. Akceptacja nastąpi, jeżeli istnieje przynajmniej jedna ważna sekwencja przejść pomiędzy stanami taka, że po przeczytaniu całego ciągu wejściowego automat znajduje się w stanie końcowym.

ϵ -NFA powstaje jeżeli pozwolimy, by w automacie NFA odbywały się przejścia pomiędzy stanami bez czytania żadnego znaku z ciągu wejściowego. Chociaż powyższe rozszerzenie nie zwiększa mocy automatu, pozwala jednak na wygodne reprezentowanie przez automat wyrażeń regularnych.

Poniżej graf automatu rozpoznającego identyfikatory i słowo kluczowe if. Wersje DFA i NFA



Dla każdego skończonego automatu deterministycznego $M_D = (Q_D; \Sigma; \delta_D; q_{0D}; F_D)$ istnieje równoważny automat niedeterministyczny $M_N = (Q_N; \Sigma; \delta_N; q_{0N}; F_N)$ rozpoznający ten sam język. Otrzymujemy go zamieniając przejścia $\delta(q; \sigma) = p$ na $\delta(q; \sigma) = \{p\}$.

Dla każdego skończonego automatu niedeterministycznego istnieje równoważny automat deterministyczny rozpoznający ten sam język. Procedura otrzymywania takiego automatu deterministycznego nazywa się determinizacją. Automat deterministyczny może mieć wykładniczo więcej stanów niż niedeterministyczny.

DETERMINIZACJA

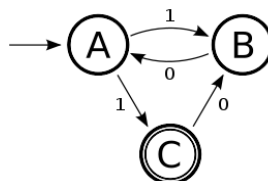
Dany jest NAS:

$S_n = \{A, B, C\}$

$\Sigma_n = \{0, 1\}$

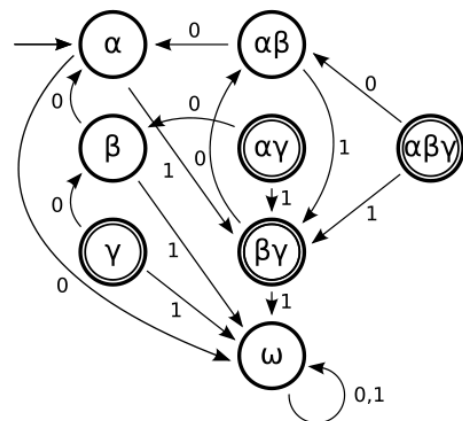
$s_n = A$

$A_n = \{C\}$



Odpowiadający mu DAS będzie miał $2^{|S_n|} = 2^3 = 8$ stanów:

- $S_{d0} = \{\alpha, \beta, \gamma, \alpha\beta, \alpha\gamma, \beta\gamma, \alpha\beta\gamma, \omega\}$
 α odpowiada stanowi A, β stanowi B, a γ stanowi C
stan $\alpha\beta$ będzie osiągany na przykład, gdy ze stanu A dla 0 na wejściu odpowiada jednocześnie przejście $A \rightarrow A$ i $A \rightarrow B$, inaczej: $A \times 0 \rightarrow \{A, B\}$
stan ω może być osiągnięty gdy dla pewnego stanu nie przewidziano przejścia dla którejś z liter z alfabetu wejściowego
- $\Sigma_{d0} = \{0, 1\}$
alfabet wejściowy nie ulega zmianie
- $s_{d0} = \alpha$
- $A_{d0} = \{\gamma, \alpha\gamma, \beta\gamma, \alpha\beta\gamma\}$
akceptujące są stany w których występuje γ odpowiadająca akceptującemu stanowi C



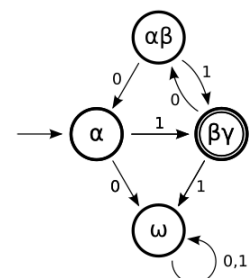
Ostatni etap polega na usunięciu stanów, których nie można osiągnąć za pomocą żadnej sekwencji liter z alfabetu wejściowego. Należy w tym celu zacząć od stanu początkowego s_{d0} i oznaczać kolejne stany do których istnieje ścieżka. Ostatecznie otrzymujemy:

$S_d = \{\alpha, \alpha\beta, \beta\gamma, \omega\}$

$\Sigma_d = \{0, 1\}$

$s_d = \alpha$

$A_d = \{\beta\gamma\}$



MINIMALIZACJA

Jako rozmiar $|M|$ automatu M przyjmuje się tradycyjnie liczbę jego stanów $|Q|$. W praktyce rozmiar automatu przechowywanego w pamięci programu zależy od liczby jego przejść. Wśród wszystkich automatów deterministycznych rozpoznających dany język jest zawsze dokładnie jeden (z dokładnością do izomorfizmu), który ma najmniejszą liczbę stanów. Automat taki nazywamy minimalnym.

Proces polegający na uzyskaniu automatu minimalnego nazywamy minimalizacją. Są trzy rodziny ogólnych algorytmów minimalizacji:

1. Stany dzielimy na dwie klasy: końcowe i niekońcowe. Następnie na podstawie przejść między stanami (do których klas stanów prowadzą przejścia lub odwrotne przejścia w danej klasie stanów) dokonujemy dalszego podziału klas. Uzyskane klasy stanowią stany automatu minimalnego. Istnieją trzy takie algorytmy:

- algorytm Hopcrofta o złożoności $O(|Q| \log |Q|)$
- algorytm Hopcroft-Ullman o złożoności $O(|Q|^2)$
- algorytm Aho-Sethi-Ullman o złożoności $O(|Q|^2)$

2. odwracamy kierunek przejść automatu, determinizujemy, ponownie odwracamy i ponownie determinizujemy — algorytm Brzozowskiego o złożoności wykładniczej.

3. porównujemy parami stany automatu, co wymaga na ogół porównywania dalszych stanów — algorytm Watsona z usprawnieniami o złożoności $O(|Q|^2 \log \log |Q|)$.

Wyrażeniem regularnym nad alfabetem Σ jest:

1. zbiór pusty $\emptyset$
2. pusty ciąg symboli ϵ (ciąg o długości zero)
3. symbol σ z alfabetu Σ
4. sklejenie $R_1 R_2$ dwóch wyrażen regularnych R_1 i R_2
5. alternatywa $R_1 | R_2$ dwóch wyrażen regularnych R_1 i R_2
6. domknięcie przechodnie R^* wyrażenia regularnego R
7. wyrażenie regularne ujęte w nawiasy
8. nic ponadto

Konstrukcja Thompsona służy do budowy automatu skończonego (ϵ -NFA) na podstawie wyrażenia regularnego. Jej cechą charakterystyczną jest to, że powstające automaty pośrednie i automat wynikowy mają szczególną postać — mają nie tylko jeden stan początkowy (jak każdy automat rozpoznający łańcuchy znaków), ale też (tylko) jeden stan końcowy. Wyrażenie rozkładane jest na podwyrażenia, a automat dla całego wyrażenia budowany jest na podstawie automatów zbudowanych dla podwyrażeń.

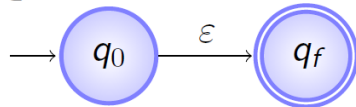
Składanie większych konstrukcji z $R_A, R_B \in RE$:

Podstawowe cegielki:

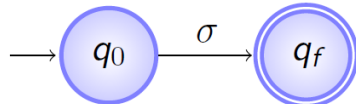
- $\emptyset \in RE$



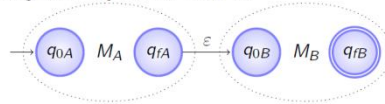
- $\epsilon \in RE$



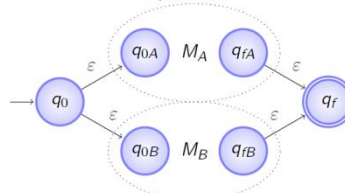
- $\sigma \in \Sigma \Rightarrow \sigma \in RE$



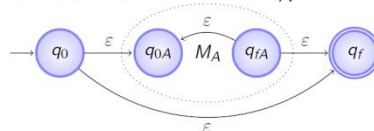
- sklejenie wyrażen $R_A R_B$



- alternatywa $R_A | R_B$

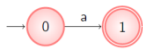


- domknięcie przechodnie R_A^*

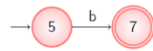
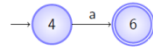
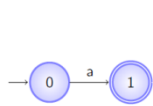


Przykład: $a(a|b)^*a$

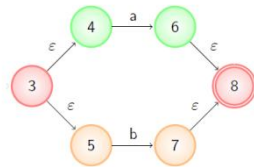
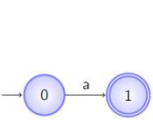
$a(a|b)^*a$



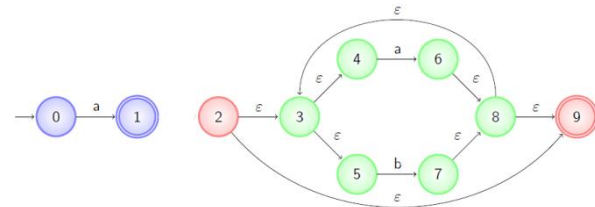
$a(a|b)^*a$



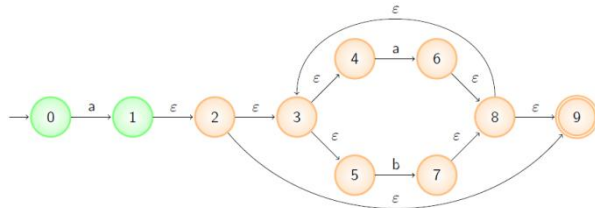
$a(a|b)^*a$



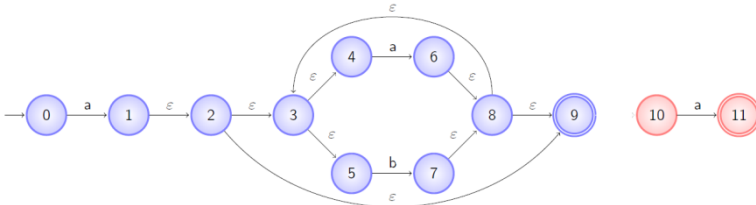
$a(a|b)^*a$



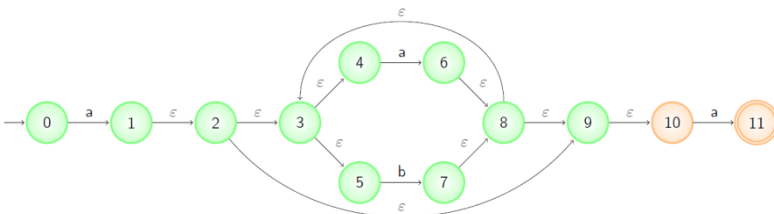
$a(a|b)^*a$



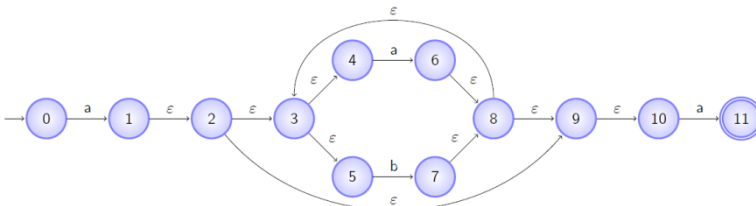
$a(a|b)^*a$



$a(a|b)^*a$



$a(a|b)^*a$



Konstrukcja McNaughton-Yamada-Głuszkow. Cecha charakterystyczną automatu będącego wynikiem tej konstrukcji jest to, że wszystkie przejścia przychodzące do danego stanu mają tę samą etykietę. Wszystkie symbole występujące w wyrażeniu regularnym są numerowane, by odróżnić różne wystąpienia tych samych symboli. Dla każdego numerowanego symbolu tworzymy stan. Tworzymy także dodatkowo stan początkowy. Przejścia i końcowość stanów określone są za pomocą 4 elementów: $\text{Null}(R)$, $\text{First}(R)$, $\text{Last}(R)$, $\text{Follow}(R)$.

Dla podstawowych elementów wyrażeń regularnych cecha Null wynosi:

- $\text{Null}(\emptyset) = \text{false}$
- $\text{Null}(\varepsilon) = \text{true}$
- $\text{Null}(\sigma \in \Sigma) = \text{false}$

Dla złożonych wyrażeń składających się z podwyrażeń f i g Null obliczamy w następujący sposób:

- $\text{Null}(fg) = \text{Null}(f) \wedge \text{Null}(g)$
- $\text{Null}(f|g) = \text{Null}(f) \vee \text{Null}(g)$
- $\text{Null}(f^*) = \text{true}$

Zbiór First dla podstawowych elementów wyrażeń regularnych obliczamy następująco:

- $\text{First}(\emptyset) = \emptyset$
- $\text{First}(\varepsilon) = \emptyset$
- $\text{First}(\sigma \in \Sigma) = \{\sigma\}$

Dla złożonych wyrażeń składających się z podwyrażeń f i g First obliczamy w następujący sposób:

- $\text{First}(fg) = \begin{cases} \text{First}(f) & \text{jeśli } \neg \text{Null}(f) \\ \text{First}(f) \cup \text{First}(g) & \text{jeśli } \text{Null}(f) \end{cases}$
- $\text{First}(f|g) = \text{First}(f) \cup \text{First}(g)$
- $\text{First}(f^*) = \text{First}(f)$

Zbiór Last dla podstawowych elementów wyrażeń regularnych obliczamy następująco:

- $\text{Last}(\emptyset) = \emptyset$
- $\text{Last}(\varepsilon) = \emptyset$
- $\text{Last}(\sigma \in \Sigma) = \{\sigma\}$

Dla złożonych wyrażeń składających się z podwyrażeń f i g Last obliczamy w następujący sposób:

- $\text{Last}(fg) = \begin{cases} \text{Last}(g) & \text{jeśli } \neg \text{Null}(g) \\ \text{Last}(f) \cup \text{Last}(g) & \text{jeśli } \text{Null}(g) \end{cases}$
- $\text{Last}(f|g) = \text{Last}(f) \cup \text{Last}(g)$
- $\text{Last}(f^*) = \text{Last}(f)$

Zbiór Follow dla podstawowych elementów wyrażeń regularnych obliczamy następująco:

- $\text{Follow}(\emptyset) = \emptyset$
- $\text{Follow}(\varepsilon) = \emptyset$
- $\text{Follow}(\sigma \in \Sigma) = \emptyset$

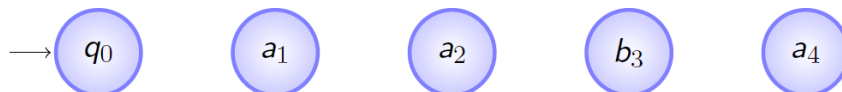
Dla złożonych wyrażeń składających się z podwyrażeń f i g Follow obliczamy w następujący sposób:

- $\text{Follow}(fg) = \text{Follow}(f) \cup \text{Follow}(g) \cup (\text{Last}(f) \times \text{First}(g))$
- $\text{Follow}(f|g) = \text{Follow}(f) \cup \text{Follow}(g)$
- $\text{Follow}(f^*) = \text{Follow}(f) \cup (\text{Last}(f) \times \text{First}(f))$

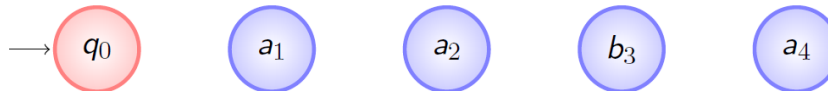
Przykład: $a(a|b)^*a$, po ponumerowaniu symboli: $a_1(a_2|b_3)^*a_4$

RE	Null	First	Last	Follow
a_1	false	$\{a_1\}$	$\{a_1\}$	$\emptyset$
a_2	false	$\{a_2\}$	$\{a_2\}$	$\emptyset$
b_3	false	$\{b_3\}$	$\{b_3\}$	$\emptyset$
$a_2 b_3$	false	$\{a_2, b_3\}$	$\{a_2, b_3\}$	$\emptyset$
$(a_2 b_3)^*$	true	$\{a_2, b_3\}$	$\{a_2, b_3\}$	$\{(a_2a_2), (a_2b_3), (b_3a_2), (b_3b_3)\}$
$a_1(a_2 b_3)^*$	false	$\{a_1\}$	$\{a_1, a_2, b_3\}$	$\{(a_1a_2), (a_1b_3), (a_2a_2), (a_2b_3), (b_3a_2), (b_3b_3)\}$
a_4	false	$\{a_4\}$	$\{a_4\}$	$\emptyset$
$a_1(a_2 b_3)^*a_4$	false	$\{a_1\}$	$\{a_4\}$	$\{(a_1a_2), (a_1b_3), (a_2a_2), (a_2b_3), (b_3a_2), (b_3b_3), (a_1a_4), (a_2a_4), (b_3a_4)\}$

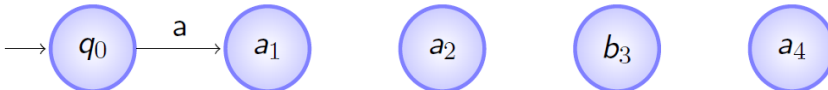
Automat ma tyle stanów, ile numerowanych symboli w wyrażeniu, plus dodatkowo stan początkowy q_0 .



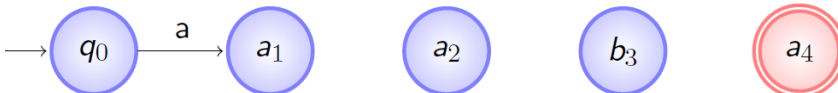
Ponieważ ϵ nie jest rozpoznawany przez to wyrażenie, to: $\text{Null}(a_1(a_2|b_3)^*a_4) = \text{false}$ i stan początkowy nie jest końcowym.



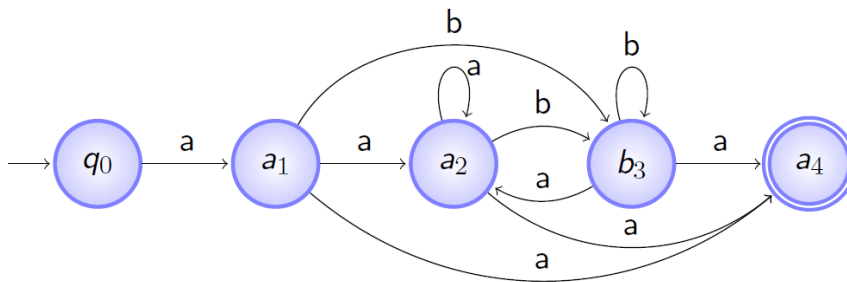
$\text{First}(a_1(a_2|b_3)^*a_4) = \{a_1\}$



$\text{Last}(a_1(a_2|b_3)^*a_4) = \{a_4\}$



$\text{Follow}(a_1(a_2|b_3)^*a_4) = \{(a_1a_2); (a_1b_3); (a_2a_2); (a_2b_3); (b_3a_2); (b_3b_3); (a_1a_4); (a_2a_4); (b_3a_4)\}$



Gramatykę bezkontekstową nazywamy czwórkę uporządkowaną $(V; \Sigma; P; S)$, gdzie:

- V to zbiór zmiennych (inaczej — symboli niekończących)
- Σ (oznaczany też jako T) to zbiór symboli końcowych (ang. terminals)
- P to zbiór reguł produkcji postaci $\alpha \rightarrow \beta$, gdzie $\alpha \in V$, zaś β to dowolny (także pusty) ciąg symboli końcowych i niekończących
- S to symbol wyróżniony — symbol początkowy gramatyki

Zapis BNF — Backus-Naur Form lub Backus Normal Form to inny zapis gramatyki bezkontekstowej:

- zmienne zapisujemy w nawiasach kątowych, np. $\langle \text{zmienna} \rangle$
- symbole końcowe zapisujemy w cudzysłowach
- ϵ zapisujemy jako $''$
- symbol przepisania ($\rightarrow$) w regule produkcji zapisujemy jako $::=$
- różne reguły produkcji z tą samą zmienną po lewej stronie można łączyć w jedną, gdzie prawe strony rozdzielone są znakiem $|$

Przykład: gramatykę $E \rightarrow 0 \mid 1 \mid E + E \mid E * E$ zapisujemy jako:

$\langle E \rangle ::= "0" \mid "1" \mid \langle E \rangle "+" \langle E \rangle \mid \langle E \rangle "*" \langle E \rangle$

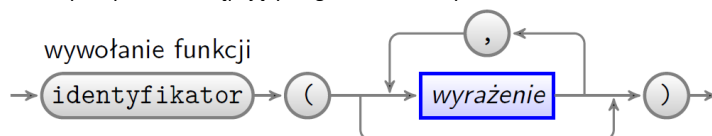
Dla gramatyki:

$\langle \text{wywołanie funkcji} \rangle ::= \text{"identyfikator" "("} \langle \text{parametry} \rangle \text{"}"$

$\langle \text{parametry} \rangle ::= "" \mid \langle \text{lista wyrażeń} \rangle$

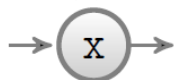
$\langle \text{lista wyrażeń} \rangle ::= \langle \text{wyrażenie} \rangle \mid \langle \text{lista wyrażeń} \rangle "," \langle \text{wyrażenie} \rangle$

możemy narysować następujący diagram składniowy:

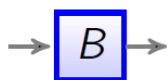


Każda reguła produkcji w zapisie BNF może zostać przekształcona na diagram składniowy o strukturze wynikającej z następujących przekształceń:

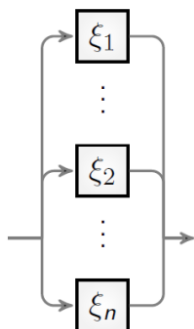
* każdy symbol końcowy x zostaje przekształcony w następujący schemat (kółko może zostać zastąpione prostokątem o zaokrąglonych bokach, gdy nazwa symbolu jest długa):



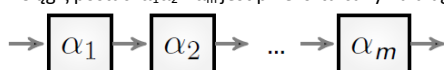
* każde wystąpienie zmiennej B jest reprezentowane przez łuk z etykietą w kwadracie:



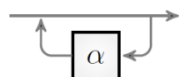
* Produkcja postaci $\langle A \rangle ::= \xi_1 \mid \dots \mid \xi_n$ jest zamieniana na poniższy diagram, w którym każde ξ_i jest zastąpione przez odpowiedni diagram składowy zgodnie z regułami:



* Ciąg ξ postaci $\alpha_1 \alpha_2 \dots \alpha_m$ jest przekształcany na diagram:



* Rozszerzenie BNF: $\{\alpha\}$ oznacza dowolną liczbę powtórzeń α . Każde $\xi = \{\alpha\}$ przekształcamy w diagram:



Ciąg przekształceń przy użyciu reguł produkcji nazywamy wyprowadzeniem.

Formą zdaniową σ gramatyki G nazywamy każde wyrażenie dające się wyprowadzić z symbolu początkowego S gramatyki G .

Drzewo wyprowadzenia dla gramatyki $G = (V; \Sigma; P; S)$ to dowolne drzewo spełniające następujące warunki:

1. każdy wierzchołek wewnętrzny etykietowany jest zmienną z V
 2. każdy liść etykietowany jest symbolem końcowym lub ϵ , przy czym liść etykietowany ϵ musi być jedynym dzieckiem swojego rodzica
 3. dla każdego wierzchołka wewnętrznego etykietowanego zmienną $A \in V$ i jego dzieci $X_1; X_2; \dots; X_k$ istnieje produkcja $A \rightarrow X_1; X_2; \dots; X_k$ w P .
- Ciąg liści drzewa wyprowadzenia nazywamy **plonem**.

Gramatyka jednoznaczna to taka gramatyka, w której dla każdego słowa należącego do języka generowanego przez tę gramatykę istnieje jedno drzewo wyprowadzenia, którego plon jest tym słowem.

Gramatyka, która nie jest jednoznaczna jest **wieloznaczna**.

Dla wielu gramatyk wieloznacznych można znaleźć gramatyki jednoznaczne, które generują ten sam język bezkontekstowy. Jednak istnieją języki, dla których wszystkie gramatyki są wieloznaczne. Są to języki **ściśle wieloznaczne**.

ANALIZA SKŁADNIOWA

Analizator składniowy otrzymuje od analizatora leksykalnego ciąg symboli końcowych i sprawdza, czy i jak można je otrzymać z symbolu początkowego gramatyki stosując reguły produkcji. Jeśli ciągu wejściowego nie da się wyprowadzić, analizator składniowy powinien wskazać przyczynę błędu i kontynuować analizę w taki sposób, by wykryć ewentualne inne błędy składniowe.

Analiza może być:

- **zstępująca** — zaczynamy od symbolu początkowego gramatyki i stosujemy reguły produkcji do czasu otrzymania ciągu symboli końcowych; ten rodzaj analizy stosowany jest głównie w analizatorach budowanych ręcznie
- **wstępująca** — zaczynamy od ciągu symboli końcowych i stosujemy redukcje ciągu symboli występujących po prawej stronie reguły produkcji zastępując je zmienną występującą po lewej stronie tej reguły produkcji; ten rodzaj analizy stosowany jest głównie w analizatorach tworzonych z użyciem narzędzi takich jak bison.

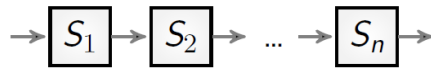
Program analizatora składniowego zstępującego można tworzyć bezpośrednio z diagramu składniowego. W praktyce stosuje się gramatyki spełniające następujące wymagania:

1. Dla każdej zadanej produkcji $A ::= \xi_1 \mid \xi_2 \mid \dots \mid \xi_n$ zbiory pierwszych symboli w zdaniach, które mogą być wyprowadzone z ξ_i muszą być rozłączne, tzn. $\text{pierw}(\xi_i) \cap \text{pierw}(\xi_j) = \emptyset$ dla wszystkich $i \neq j$
 2. Dla każdego symbolu $A \in V$, z którego można wyprowadzić pusty ciąg symboli, tzn. $A \Rightarrow^* \epsilon$, zbiór jego pierwszych symboli musi być rozłączny ze zbiorem symboli, które mogą występować po dowolnym ciągu wyprowadzonym z A , tzn. $\text{pierw}(A) \cap \text{nast}(A) = \emptyset$
- Gramatyka taka nazywa się $LL(1)$, gdzie pierwsze L pochodzi od analizy od lewej (Left), drugie od lewostronnego (Leftmost) wyprowadzenia, a 1 od podglądu 1 symbolu.

Analiza zstępująca — metoda zejść rekurencyjnych

Konstrukcja analizatora zstępującego dla diagramu składniowego przebiega według następujących reguł, w których $T(S)$ oznacza instrukcję otrzymaną z diagramu S :

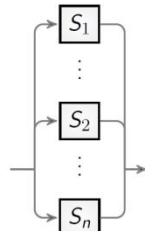
1. Zredukuj, na ile to możliwe, liczbę diagramów, stosując odpowiednie podstawienia.
2. Każdy diagram zastąp deklaracją procedury zgodnie z dalszymi regułami.
3. Ciąg elementów postaci:



jest przekształcany na instrukcję postaci

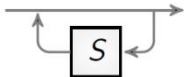
```
{ T(S1); T(S2); ... ; T(Sn); }
```

4. Diagram alternatywy jest zamieniany na instrukcję wyboru gdzie L_i oznacza zbiór symboli początkowych konstrukcji S_i^1 , tzn. $L^1 = \text{pierz}(S_i)$. Oczywiście zamiast instrukcji wyboru można użyć instrukcji warunkowej.



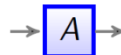
```
switch (ch) {  
  case L1: T(S1);  
  case L2: T(S2);  
  ...  
  case Ln: T(Sn);  
}
```

5. Diagram pętli zamieniamy na instrukcję while gdzie $L = \text{pierz}(S)$, zaś in przynależność do zbioru.



```
while (ch in L) T(S);
```

6. Element diagramu oznaczający inny diagram A zastępujemy wywołaniem procedury A



```
A();
```

7. Element diagramu oznaczający symbol końcowy x zastępujemy poniższą instrukcją



```
if (ch == x) ch = getc(); else błąd;
```

Analiza zstępująca — lewostronna rekurencja

Gramatyka LL(1) wprowadza ograniczenia na postać reguł produkcyjnych. Jeśli by tych ograniczeń nie było, mogłoby to spowodować kłopoty. Kłopoty może sprawiać np. lewostronna rekurencja. Rozważmy regułę:

$\text{lista} \rightarrow \text{lista ' ' element} \mid \text{element}$

Na wejściu mamy element. Rozwijamy lista używając pierwszego wariantu i otrzymując lista ' ' element. Na pierwszym miejscu mamy znów lista. Rozwijamy go i znów na pierwszym miejscu mamy lista...

W regule są dwa warianty, ale w pierwszym wariantcie element należy do $\text{pierz}(\text{lista})$. W tym przypadku można zamienić rekurencję na prawostronną, która nie sprawia kłopotu dla analizy zstępującej metodą zejść rekurencyjnych:

$\text{lista} \rightarrow \text{element ' ' lista} \mid \text{element}$

Usuwanie lewostronnej rekurencji w gramatyce LL(1)

Rozważmy regułę postaci:

$A \rightarrow A\alpha \mid \beta$

gdzie $\alpha, \beta \in (\Sigma \cup V)$ to takie ciągi zmiennych i symboli końcowych, które nie zaczynają się od A . Lewostronną rekurencję można usunąć przepisując reguły następująco:

$A \rightarrow \beta R$

$R \rightarrow \alpha R \mid \epsilon$

Analiza zstępująca z jawnym stosem

Rozważmy uproszczoną gramatykę wyrażeń arytmetycznych:

$$E \rightarrow E + T \mid T$$

$$T \rightarrow T * F \mid F$$

$$F \rightarrow (E) \mid \text{id}$$

Po usunięciu lewostronnej rekurencji otrzymamy:

$$E \rightarrow TE'$$

$$E' \rightarrow +TE' \mid \epsilon$$

$$T \rightarrow FT'$$

$$T' \rightarrow *FT' \mid \epsilon$$

$$F \rightarrow (E) \mid \text{id}$$

Obliczmy zbiory $\text{pierw}(a)$ i $\text{nast}(A)$, a następnie wypełnijmy tablicę analizatora.

Gramatyka

$$E \rightarrow TE'$$

$$E' \rightarrow +TE' \mid \epsilon$$

$$T \rightarrow \textcolor{red}{FT'}$$

$$T' \rightarrow *FT' \mid \epsilon$$

$$F \rightarrow (E) \mid \text{id}$$

$\text{pierw}(A)$

$$E \mapsto \{ (, \text{id} \}$$

$$E' \mapsto \{ +, \epsilon \}$$

$$T \mapsto \{ (, \text{id} \}$$

$$T' \mapsto \{ *, \epsilon \}$$

$$F \mapsto \{ (, \text{id} \}$$

$\text{nast}(A)$

$$E \mapsto \{ \$,) \}$$

$$E' \mapsto \{ \$,) \}$$

$$T \mapsto \{ +,), \$ \}$$

$$\textcolor{red}{T'} \mapsto \{ +,), \$ \}$$

$$\textcolor{red}{F} \mapsto \{ *, +,), \$ \}$$

Obliczanie $\text{pierw}(A)$

- 1 Jeśli pierwszy symbol argumentu jest symbolem końcowym a , to $\text{pierw}(a\xi) = \{a\}$
- 2 Jeśli $\xi \rightarrow \epsilon$ jest produkcją, to należy dodać ϵ do $\text{pierw}(\xi)$
- 3 Jeśli ξ jest zmienną i $\xi \rightarrow \xi_1 \xi_2 \dots \xi_k$ jest produkcją, to a należy umieścić w $\text{pierw}(\xi)$, jeśli istnieje takie i , że a jest w $\text{pierw}(\xi_i)$, a ϵ jest we wszystkich $\text{pierw}(\xi_1), \dots, \text{pierw}(\xi_{i-1})$. Jeśli $\epsilon \in \text{pierw}(\xi_i)$ dla wszystkich $j = 1, \dots, k$, to do $\text{pierw}(\xi)$ należy dodać ϵ .

Obliczanie $\text{nast}(A)$

- 1 W $\text{nast}(S)$, gdzie S jest symbolem początkowym, należy umieścić znacznik końca wejścia $\$$.
- 2 Jeśli mamy produkcję $A \rightarrow \alpha B \beta$, to wszystkie symbole z $\text{pierw}(\beta)$ z wyjątkiem ϵ należy umieścić w $\text{nast}(B)$.
- 3 Jeśli mamy produkcję $A \rightarrow \alpha B$ albo produkcję $A \rightarrow \alpha B \beta$, gdzie $\epsilon \in \text{pierw}(\beta)$, to wszystkie symbole z $\text{nast}(A)$ są w $\text{nast}(B)$.

Wypełnianie tablicy analizatora:

Tablica analizatora

zm.	symbol wejściowy					
	id	+	*	(	)	\$
E	$E \rightarrow TE'$			$E \rightarrow TE'$		
E'		$E' \rightarrow +TE'$			$E' \rightarrow \varepsilon$	$E' \rightarrow \varepsilon$
T	$T \rightarrow FT'$			$T \rightarrow FT'$		
T'		$T' \rightarrow \varepsilon$	$T' \rightarrow *FT'$		$T' \rightarrow \varepsilon$	$T' \rightarrow \varepsilon$
F	$F \rightarrow \text{id}$			$F \rightarrow (E)$		

Algorytm wypełniania tablicy analizatora

```

1: for all reguła produkcji  $A \rightarrow \alpha$  gramatyki do
2:   for all  $a \in \Sigma : a \in \text{pierw}(\alpha)$  do
3:     dodaj  $A \rightarrow \alpha$  do  $M[A, a]$ 
4:   end for
5:   if  $\varepsilon \in \text{pierw}(\alpha)$  then
6:     for all  $b \in \Sigma : b \in \text{nast}(A)$  do
7:       dodaj  $A \rightarrow \alpha$  do  $M[A, b]$ 
8:     end for
9:     if  $\$ \in \text{nast}(A)$  then
10:      dodaj  $A \rightarrow \alpha$  do  $M[A, \$]$ 
11:    end if
12:  end if
13: end for
14: Wstaw błąd na każdą niezdefiniowaną pozycję  $M$ 

```

Gramatyka

$E \rightarrow TE'$
 $E' \rightarrow +TE' | \varepsilon$
 $T \rightarrow FT'$
 $T' \rightarrow *FT' | \varepsilon$
 $F \rightarrow (E) | \text{id}$

$\text{pierw}(A)$

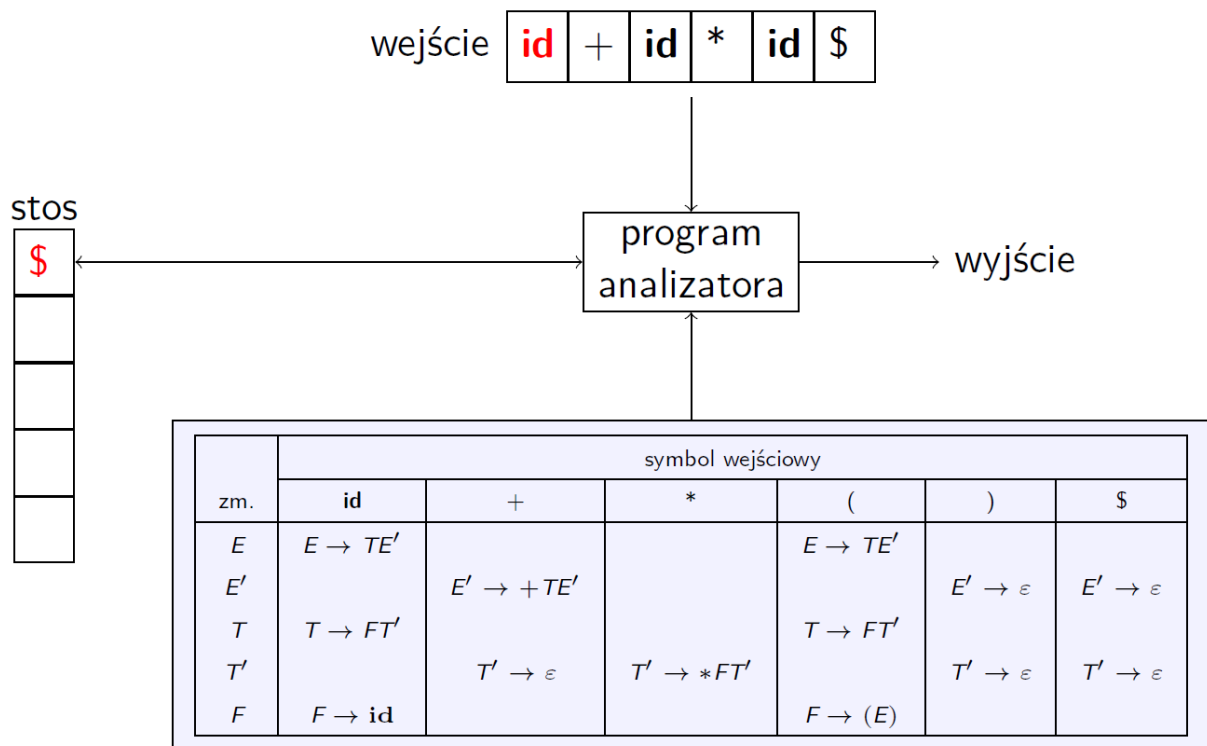
$E \mapsto \{ (, \text{id} \}$
 $E' \mapsto \{ +, \varepsilon \}$
 $T \mapsto \{ (, \text{id} \}$
 $T' \mapsto \{ *, \varepsilon \}$
 $F \mapsto \{ (, \text{id} \}$

$\text{nast}(A)$

$E \mapsto \{ \$,) \}$
 $E' \mapsto \{ \$,) \}$
 $T \mapsto \{ +,), \$ \}$
 $T' \mapsto \{ +,), \$ \}$
 $F \mapsto \{ *, +,), \$ \}$

Ten sam algorytm, ale w wersji bardziej przyjaznej:

1. Dla każdej produkcji $A \rightarrow \alpha$ z gramatyki wykonaj kroki 2. i 3.
2. Dla każdego nieterminala a z $\text{FIRST}(\alpha)$ dodaj $A \rightarrow \alpha$ do $M[A, a]$.
3. Jeśli ε jest w $\text{FIRST}(\alpha)$, dodaj $A \rightarrow \alpha$ do $M[A, b]$ dla każdego terminala b z $\text{FOLLOW}(A)$. Jeśli ε jest w $\text{FIRST}(\alpha)$ oraz $\$$ jest w $\text{FOLLOW}(A)$, dodaj $A \rightarrow \alpha$ do $M[A, \$]$.
4. Wstaw **błąd** na każdą niezdefiniowaną pozycję M . □



Analizator przewidujący sterowany tablicami składa się z bufora wejściowego, stosu, tablicy analizatora składniowego i strumienia wyjściowego. Bufor wejściowy zawiera ciąg symboli do zanalizowania, zakończony symbolem \$, używanym jako znacznik prawego końca tekstu do zaznaczania końca ciągu wejściowego. Na stosie są przechowywane symbole gramatyki, z \$ na dole, oznaczającym koniec stosu. Początkowo na stosie jest tylko startowy symbol gramatyki, leżący nad \$. Tablica analizatora jest dwuwymiarową tablicą $M[A, a]$, gdzie A jest nieterminalem, a a jest terminalem bądź symbolem \$.

Analizator jest sterowany przez program, który zachowuje się następująco: rozważa X , symbol na wierzchołku stosu, oraz a , aktualny symbol na wejściu. Te dwa symbole determinują czynność wykonywaną przez analizator. Istnieją trzy możliwości.

1. Jeśli $X = a = \$$, to analizator zatrzymuje się i ogłasza poprawne zakończenie analizy składniowej.
2. Jeśli $X = a \neq \$$, to analizator zdejmuje X ze stosu i przesuwa wskaźnik wejścia do następnego symbolu wejściowego.
3. Jeśli X jest nieterminalem, to program sprawdza wartość $M[X, a]$ w tablicy analizatora M . Tą wartością będzie albo X -produkcja gramatyki, albo wartość **błąd**. Jeśli, na przykład, $M[X, a] = \{X \rightarrow UVW\}$, to analizator zastępuje X z wierzchołka stosu przez WVU (z U na wierzchołku). Przyjmijmy, że wyjściem analizatora jest wypisanie użytej produkcji; można w tym miejscu wykonać dowolny kod. Jeśli $M[X, a] = \text{błąd}$, analizator wywołuje procedurę obsługi błędu.

Zachowanie analizatora może być opisane terminami jego *konfiguracji*, które określają zawartość stosu i pozostające wejście.

Analiza wstępująca

Analiza wstępująca polega na redukowaniu ciągów symboli do pojedynczych symboli używając reguł produkcji gramatyki aż do osiągnięcia w ten sposób symbolu początkowego gramatyki.

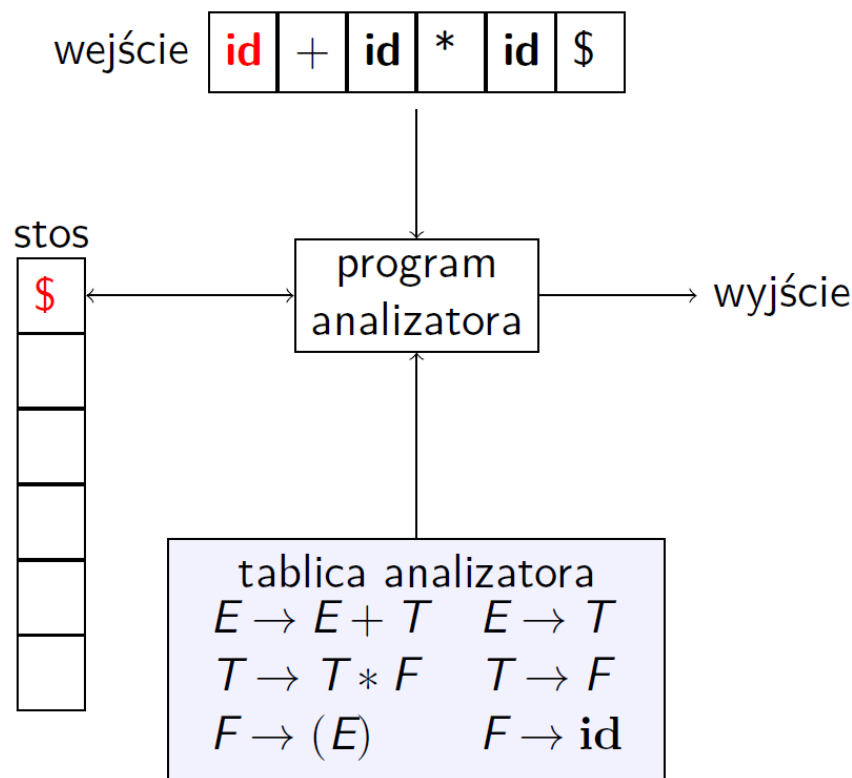
Rozpatrzmy ogólną zasadę działania takiego analizatora bez wchodzenia w szczegóły. Przyjmijmy w naszym przykładzie tę samą gramatykę, co w przykładzie analizy zstępującej z jawnym stosem (przed przekształceniem usuwającym lewostronną rekurencję):

$E \rightarrow E + T \mid T$

$T \rightarrow T * F \mid F$

$F \rightarrow (E) \mid \text{id}$

Analizator wykonuje dwa możliwe działania: **przesunięcie** symbolu z wejścia na stos i **redukcję** (czyli po polsku zwinięcie) ciągu symboli na stosie polegające na zastąpieniu ciągu występującego po prawej stronie reguły produkcji przez zmienną po lewej stronie tej reguły.



Implementacja stosowa analizy redukującej

Są dwa problemy, które musimy rozwiązać, jeśli chcemy analizować teksty, używając przycinania uchwytów. Pierwszym jest znalezienie podciągu, który trzeba zredukować, w prawostronnej formie zdaniowej, a drugim – wybranie jednej z produkcji, gdy jest więcej niż jedna produkcja z tym podciągiem po prawej stronie. Zanim zaczniemy rozwiązywać te problemy, rozważmy, jakich rodzajów struktur danych można używać w analizatorze redukującym.

Wygodną metodą implementacji analizatora redukującego jest użycie stosu do pamiętania symboli gramatyki oraz bufora wejściowego do przechowywania tekstu w , który chcemy przeanalizować. Używamy $\$$ do zaznaczenia dna stosu oraz prawego końca wejścia. Początkowo stos jest pusty, a na wejściu jest napis w

STOS	WEJŚCIE
\$	$w\$$

Analizator przesuwà zero lub więcej symboli z wejścia na stos, aż na wierzchołku stosu będzie uchwyt β . Analizator redukuje wtedy β do lewej strony odpowiedniej produkcji. Powtarza on ten cykl aż do wystąpienia błędu albo do czasu, gdy na stosie będzie symbol startowy, a wejście będzie puste

W powyższym przykładzie analizator zatrzyma się po osiągnięciu konfiguracji: $\$E$ na stosie i $\$$ na wejściu;

Analiza wstępująca LR(k)

Istnieją trzy podstawowe typy analizatorów LR(1), ale wszystkie trzy korzystają z tablicy, która określa działanie analizatora na podstawie symbolu na szczycie stosu i następnego symbolu na wejściu. Są dostępne cztery działania:

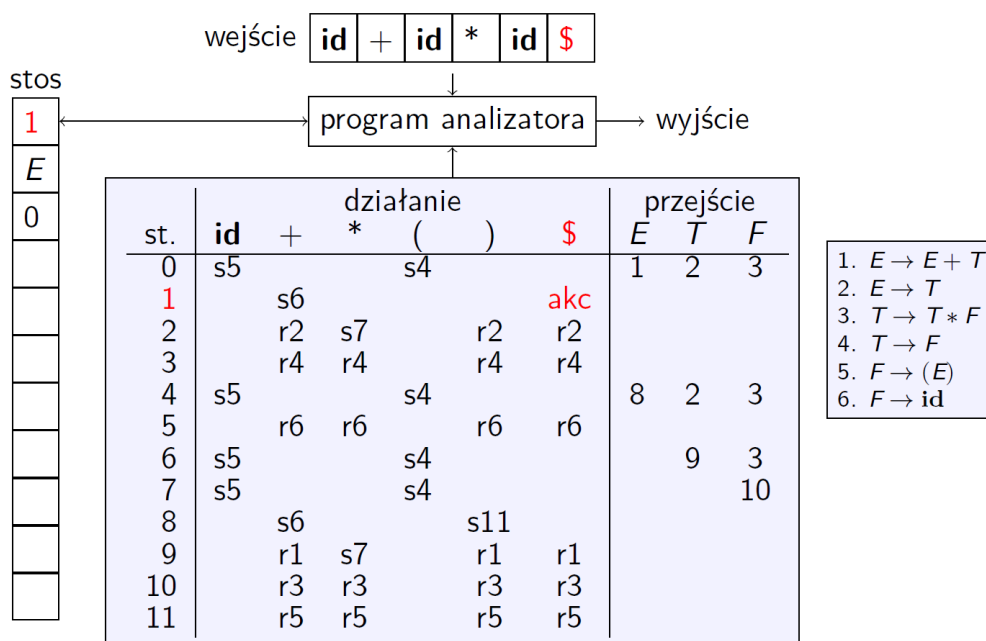
1. przeniesienie (przesunięcie) symbolu na stos (ang. shift)
2. zwinięcie kilku symboli na szczycie stosu (ang. reduce) przez zastąpienie ich lewą stroną wskazanej reguły produkcji, której prawą stroną stanowią
3. przyjęcie wejścia
4. odrzucenie wejścia (błąd)

Na stos odkładamy parę (symbol, stan). Stan stanowi numer wiersza w tablicy analizatora. Początkowo na stosie znajduje się numer 0.

Wiersze tablicy analizatora oznaczają stany. Kolejne kolumny tablicy analizatora należą do dwóch grup: działanie i przejście. Kolumny działania etykietowane są symbolami końcowymi (symbolami z alfabetu) i znakiem końca wejścia \$. Zawartość tych kolumn oznaczamy jako literę s lub r i liczbę, przy czym:

- dla przesunięcia mamy literę s, a liczba jest numerem stanu docelowego — wiersza w tablicy analizatora
- dla zwinięcia mamy literę r, a liczba jest numerem reguły produkcji, według której zwijamy stos

Kolumny przejścia etykietowane są zmiennymi gramatyki i zawierają numery stanów docelowych. Etykiety interpretowane są jako lewe strony reguł produkcji, według których następuje zwijanie.



Algorytm analizy LR(k)

```

1: repeat
2:   niech a będzie bieżącym symbolem na wejściu
3:   niech s będzie symbolem z wierzchołka stosu
4:   if działanie[s, a] = przesun s' then
5:     odłóż a na stos
6:     odłóż s' na stos
7:     przesun wskaźnik wejścia, ustaw a na nast. symbol na wejściu
8:   else if działanie[s, a] = zwinięcie według reguły  $A \rightarrow \beta$  then
9:     zdejmij ze stosu  $|\beta|$  par (symbol, nr stanu)
10:    niech s' będzie stanem na szczycie stosu
11:    odłóż A na stos
12:    odłóż przejście[s', a] na stos
13:   else if działanie[a, s] = przyjmij then
14:     zakończ działanie — rozpoznano wejście
15:   else
16:     zgłoś błąd
17:   end if
18: until True
  
```

Kolejny ruch analizatora jest wyznaczany przez odczytanie a_i , aktualnego symbolu wejściowego, i s_m — stanu na wierzchołku stosu, a następnie sprawdzenie wartości w tablicy akcji analizatora $akcja[s_m, a_i]$. Konfiguracje, które mogą wystąpić po każdym z czterech typów akcji, to:

1. Jeśli $akcja[s_m, a_i] =$ przesun s , analizator wykonuje przesunięcie, przechodząc do konfiguracji

$$(s_0 X_1 s_1 X_2 s_2 \cdots X_m s_m a_i s, a_{i+1} \cdots a_n \$)$$

Analizator wstawił na stos aktualny symbol wejściowy a_i i następny stan s , który jest pobierany z $akcja[s_m, a_i]$; a_{i+1} staje się aktualnym symbolem wejściowym.

2. Jeśli $akcja[s_m, a_i] =$ redukuj według $A \rightarrow \beta$, to analizator wykonuje redukcję, przechodząc do konfiguracji

$$(s_0 X_1 s_1 X_2 s_2 \cdots X_{m-r} s_{m-r} A s, a_i a_{i+1} \cdots a_n \$)$$

gdzie $s = przejście[s_{m-r}, A]$, a r jest długością β — prawej strony produkcji. Analizator zdejmie ze stosu $2r$ symboli (r symboli stanu i r symboli gramatyki), odkrywając stan s_{m-r} . Następnie wstawia na stos A — lewą stronę użytej produkcji i s — wartość $przejście[s_{m-r}, A]$. Aktualny symbol wejściowy w wyniku redukcji nie jest zmieniany. Dla analizatorów LR, które będziemy budowali, ciąg symboli gramatyki zdjętych ze stosu, $X_{m-r+1} \cdots X_m$, zawsze będzie pasował do β — prawej strony produkcji używanej do redukcji.

Wyjście analizatora LR jest generowane po wykonaniu redukcji przez wykonanie akcji semantycznej związanej z redukowaną produkcją. Chwilowo przyjmujemy, że wyjściem jest tylko wypisanie używanej produkcji.

3. Jeśli $akcja[s_m, a_i] =$ akceptuj, analiza jest zakończona.
4. Jeśli $akcja[s_m, a_i] =$ błąd, analizator wykrył błąd i wywołuje procedurę obsługi błędu.

Istnieją różne rodzaje analizatorów LR(1). Różnią się sposobem wypełniania tablicy i jej rozmiarem, a także złożonością wypełniania i klasą rozpoznawanych języków. Rozpatrzmy: Simple LR (SLR), Kanoniczna LR i Look-Ahead LR (LARL). Mówimy też o gramatykach i językach SLR i LARL.

Analiza SLR

Tablica analizatora SLR powstaje jako niedeterministyczny automat skończony z przejściami etykietowanymi symbolem ϵ (pustym ciągiem symboli). Stanami są sytuacje gramatyki.

Sytuacją LR(0) (w skrócie *sytuacją*) gramatyki G nazywamy produkcję G z kropką w jakimś miejscu jej prawej strony. Z produkcji $A \rightarrow XYZ$, na przykład, otrzymujemy cztery sytuacje

$$\begin{aligned} A &\rightarrow \cdot XYZ \\ A &\rightarrow X \cdot YZ \\ A &\rightarrow XY \cdot Z \\ A &\rightarrow XYZ \cdot \end{aligned}$$

Z produkcji $A \rightarrow \epsilon$ mamy tylko jedną sytuację, $A \rightarrow \cdot$. Sytuację możemy reprezentować parą liczb, z których pierwsza reprezentuje numer produkcji, a druga pozycję kropki. Intuicyjnie, sytuacja wskazuje, jaki fragment produkcji już widzieliśmy w danym punkcie procesu wyprowadzania. Przykładowo, pierwsza z powyższych sytuacji oznacza, że spodziewamy się na wejściu znaleźć ciąg wyprowadzalny z XYZ . Druga sytuacja oznacza, że odczytaliśmy już ciąg wyprowadzalny z X i teraz oczekujemy ciągu wyprowadzalnego z YZ .

Ważne w metodzie SLR jest to, że rozpoczynamy od konstruowania z gramatyki deterministycznego automatu skończonego, rozpoznającego prefiksy żywotne. Sytuacje łączymy w zbiory, z których powstają stany analizatora SLR. Sytuacje można traktować

jak stany niedeterministycznego automatu skończonego rozpoznającego prefiksy żywotne, a „łączenie w zbiory” jest w istocie budową podzbiorów opisaną w p. 3.6.

Pewna rodzina zbiorów sytuacji LR(0), którą będziemy nazywać *kanoniczną* rodziną LR(0) dla gramatyki, zapewnia podstawę do konstrukcji analizatorów SLR. Aby wyznaczyć kanoniczną rodzinę LR(0) dla gramatyki, definiujemy wzbogaconą gramatykę oraz dwie funkcje, *domknięcie* i *przejście*.

Jeśli G jest gramatyką o symbolu startowym S , to G' , *wzbogaconą gramatykę* G , definiujemy jako G z nowym symbolem startowym S' i produkcją $S' \rightarrow S$. Tę nową produkcję startową wstawiamy po to, by analizator zauważył, kiedy należy skończyć analizę. Oznacza to, że wejście jest akceptowane wtedy i tylko wtedy, gdy analizator ma właśnie zredukować $S' \rightarrow S$.

Operacja domknięcia

Jeśli I jest zbiorem sytuacji gramatyki G , to $\text{domknięcie}(I)$ jest zbiorem sytuacji otrzymanych z I przy zastosowaniu dwóch reguł:

1. Początkowo, każda sytuacja z I jest dodawana do $\text{domknięcie}(I)$.
2. Jeśli $A \rightarrow \alpha \cdot B\beta$ jest w $\text{domknięcie}(I)$, a $B \rightarrow \gamma$ jest produkcją, to — jeśli nie zrobiliśmy tego wcześniej — do I dodajemy sytuację $B \rightarrow \cdot \gamma$. Stosujemy tę regułę aż do $\text{domknięcie}(I)$ nie będziemy mogli dodać żadnych nowych elementów.

Intuicyjnie, gdy $A \rightarrow \alpha \cdot B\beta$ jest w $\text{domknięcie}(I)$, to znaczy, że w pewnej chwili podczas prowadzenia analizy oczekujemy, iż na wejściu może być podciąg wyprowadzalny z $B\beta$. Jeśli mamy produkcję $B \rightarrow \gamma$, to oczekujemy również, że w tej samej chwili na wejściu może być podciąg wyprowadzalny z γ . Z tego powodu do $\text{domknięcie}(I)$ dołączamy $B \rightarrow \cdot \gamma$.

Przykład 4.34. Rozważmy wzbogaconą gramatykę dla wyrażeń:

$$\begin{aligned} E' &\rightarrow E \\ E &\rightarrow E + T \mid T \\ T &\rightarrow T * F \mid F \\ F &\rightarrow (E) \mid \text{id} \end{aligned} \tag{4.19}$$

Jeśli I jest jednoelementowym zbiorem $\{[E' \rightarrow \cdot E]\}$, to $\text{domknięcie}(I)$ zawiera następujące sytuacje:

$$\begin{aligned} E' &\rightarrow \cdot E \\ E &\rightarrow \cdot E + T \\ E &\rightarrow \cdot T \\ T &\rightarrow \cdot T * F \\ T &\rightarrow \cdot F \\ F &\rightarrow \cdot (E) \\ F &\rightarrow \cdot \text{id} \end{aligned}$$

Sytuacja $E' \rightarrow \cdot E$ jest umieszczana w $\text{domknięcie}(I)$ zgodnie z regułą 1. Ponieważ E jest bezpośrednio za kropką, więc zgodnie z regułą 2. dodajemy E -produkcje z kropkami po lewej stronie, czyli $E \rightarrow \cdot E + T$ i $E \rightarrow \cdot T$. Następnie, dlatego że T jest bezpośrednio

po kropce, dodajemy $T \rightarrow \cdot T * F$ i $T \rightarrow \cdot F$. Teraz F jest po prawej stronie kropki, co powoduje dodanie $F \rightarrow \cdot (E)$ i $F \rightarrow \cdot \text{id}$. Używając reguł 1. i 2. do *domknięcia*(I) nie można dodać żadnych innych sytuacji. $\square$

Funkcja *domknięcie* może być obliczana tak, jak pokazano na rys. 4.33. Wygodną metodą implementacji funkcji *domknięcie* jest utrzymywanie tablicy *dodane* zmiennych booleowskich, indeksowanej nieterminalami z G , takiej, że *dodane*[B] jest nadawana wartość **true** wtedy, gdy dodajemy sytuację $B \rightarrow \cdot \gamma$ dla każdej B -produkcji $B \rightarrow \gamma$.

```
function domknięcie(I);
begin
  J := I;
  repeat
    for każdej sytuacji  $A \rightarrow \alpha \cdot B \beta$  z J oraz każdej produkcji
       $B \rightarrow \gamma$  z G takiej, że  $B \rightarrow \cdot \gamma$  nie jest w J do
      dodaj  $B \rightarrow \cdot \gamma$  do J;
  until do J nie można dodać nowych sytuacji;
  return J
end
```

Rys. 4.33. Obliczanie *domknięcia*

Warto zauważyć, że jeśli jedna B -produkcja jest dodawana do domknięcia I z kropką na lewym krańcu, to analogicznie są dodawane wszystkie B -produkcyjne. W pewnych okolicznościach nie trzeba nawet zapamiętywać wszystkich sytuacji o postaci $B \rightarrow \cdot \gamma$ dodanych do I przez *domknięcie*. Wystarczy wówczas pamiętać listę nieterminali B , których produkcje dodano w ten sposób. Okazuje się, że możemy podzielić każdy z interesujących nas zbiorów sytuacji na dwie klasy sytuacji:

1. *Jądro zbioru sytuacji*, którym są wszystkie sytuacje, których kropka nie jest na lewym krańcu oraz sytuacja startowa $S' \rightarrow \cdot S$.
2. *Sytuacje spoza jądra*, czyli sytuacje, które mają kropkę na lewym krańcu.

Co więcej, każdy zbiór sytuacji, który nas interesuje, jest tworzony przez domknięcie jądra zbioru sytuacji; sytuacje, które dodajemy podczas domykania nie mogą, oczywiście, należeć do jądra. Możemy więc pamiętać zbiory sytuacji, które nas interesują, korzystając z bardzo małej ilości pamięci, jeżeli zdecydujemy się na odrzucenie wszystkich sytuacji spoza jądra, wiedząc, że mogą one zostać odtworzone przy użyciu operacji domknięcia.

Operacja przejścia

Drugą wygodną funkcją jest *przejście*(I, X), gdzie I jest zbiorem sytuacji, a X jest symbolem z gramatyki. *przejście*(I, X) jest definiowane jako domknięcie zbioru wszystkich sytuacji $[A \rightarrow \alpha X \beta]$ takich, że $[A \rightarrow \alpha \cdot X \beta]$ jest w I . Intuicyjnie, jeśli I jest zbiorem sytuacji, które są możliwe dla pewnego prefiksu żywotnego γ , to *przejście*(I, X) jest zbiorem sytuacji, które są możliwe dla prefiksu żywotnego γX .

Przykład 4.35. Jeśli I jest zbiorem dwóch sytuacji, $\{[E' \rightarrow E \cdot]$ oraz $[E \rightarrow E \cdot + T]\}$, to $\text{przejście}(I, +)$ składa się z

$$\begin{aligned} E &\rightarrow E + \cdot T \\ T &\rightarrow \cdot T * F \\ T &\rightarrow \cdot F \\ F &\rightarrow \cdot (E) \\ F &\rightarrow \cdot \text{id} \end{aligned}$$

Obliczyliśmy $\text{przejście}(I, +)$, wyszukując w I sytuacje, które miały $+$ bezpośrednio po prawej stronie kropki. $E' \rightarrow E \cdot$ nie jest taką sytuacją, ale $E \rightarrow E \cdot + T$ jest. Przesuwamy kropkę za $+$, otrzymując $\{E \rightarrow E + \cdot T\}$, i obliczamy domknięcie tego zbioru.

Tworzenie automatu dla tablicy SLR

- 1: Z gramatyki $G = (\Sigma, V, P, S)$ utwórz gramatykę $G' = (\Sigma, V \cup \{S'\}, P \cup \{S' \rightarrow S\}, S')$
- 2: Utwórz stany automatu z sytuacji G'
- 3: Każdą sytuację $A \rightarrow \alpha \cdot B \beta$, $\alpha, \beta \in (\Sigma \cup V)^*$, $B \in (\Sigma \cup V)$ połącz z sytuacją $A \rightarrow \alpha B \cdot \beta$ przejściem etykietowanym B
- 4: Każdą sytuację $A \rightarrow \alpha \cdot B \beta$ połącz z sytuacją $B \rightarrow \cdot \gamma$, $\gamma \in (\Sigma \cup V)^*$ przejściem etykietowanym ε
- 5: Determinizuj automat

Algorytm wypełniania tablicy analizatora SLR

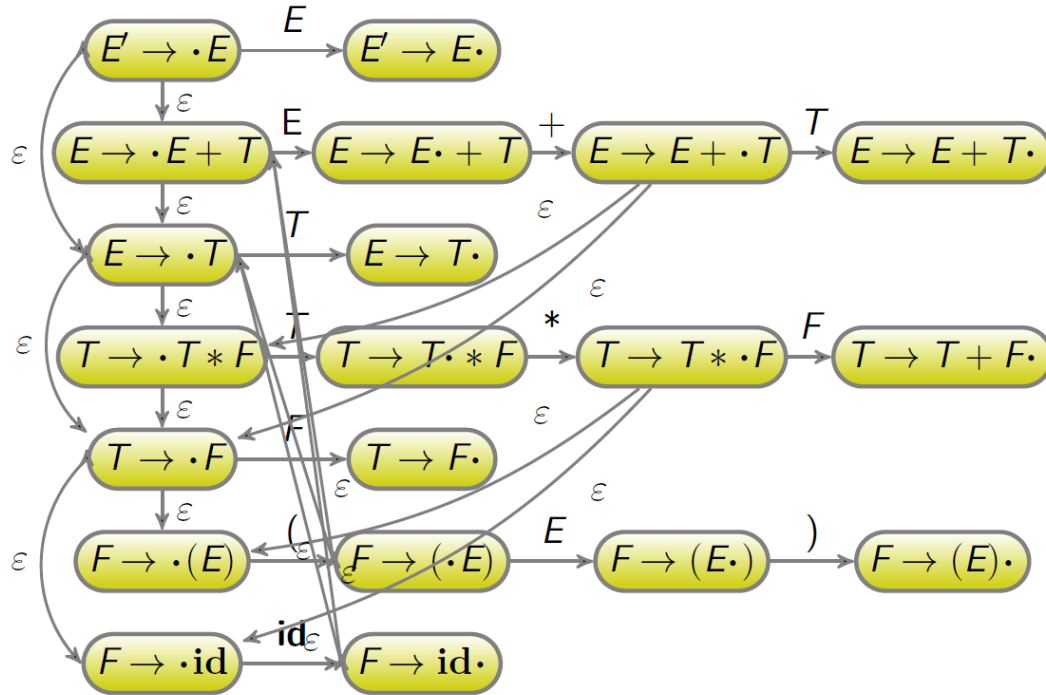
- 1: Utwórz automat deterministyczny ze stanami $l_0, l_1, \dots, l_n$
- 2: **if** $A \rightarrow \alpha \cdot a \beta$ dla $a \in \Sigma$ jest w l_i i $\delta(l_i, a) = l_j$ **then**
- 3: działanie $[i, a] = s_j$
- 4: **end if**
- 5: **if** $A \rightarrow \alpha \cdot$ jest w l_i , $A \neq S'$ **then**
- 6: **for all** $a \in \text{nast}(A)$ **do**
- 7: działanie $[i, a] = r_m$, gdzie m jest numerem produkcji $A \rightarrow \alpha$
- 8: **end for**
- 9: **end if**
- 10: **if** $S' \rightarrow S \cdot$ jest w l_i **then**
- 11: działanie $[i, \$] = \text{akc}$
- 12: **end if**
- 13: **for all** A takie że $\delta(l_i, A) = l_j$ **do**
- 14: przejście $[i, A] = j$
- 15: **end for**

Przykład. Weźmy jeszcze raz gramatykę dla wyrażeń arytmetycznych:

1. $E \rightarrow E + T$
2. $E \rightarrow T$
3. $T \rightarrow T * F$
4. $T \rightarrow F$
5. $F \rightarrow (E)$
6. $F \rightarrow \text{id}$

uzupełniamy gramatykę o dodatkową regułę produkcji:

0. $E' \rightarrow E$



l_0

$E' \rightarrow \cdot E$
 $E \rightarrow \cdot E + T$
 $E \rightarrow \cdot T$
 $T \rightarrow \cdot T * F$
 $T \rightarrow \cdot F$
 $F \rightarrow \cdot (E)$
 $F \rightarrow \cdot \text{id}$

l_1

$E' \rightarrow E \cdot$
 $E \rightarrow E \cdot + T$

l_2

$E \rightarrow T \cdot$
 $T \rightarrow T \cdot * F$

l_3

$T \rightarrow F \cdot$

l_4

$F \rightarrow (\cdot E)$
 $E \rightarrow \cdot E + T$
 $E \rightarrow \cdot T$
 $T \rightarrow \cdot T * F$
 $T \rightarrow \cdot F$
 $F \rightarrow \cdot (E)$
 $F \rightarrow \cdot \text{id}$

l_5

$F \rightarrow \text{id} \cdot$

l_6

$E \rightarrow E + \cdot T$
 $T \rightarrow \cdot T * F$
 $T \rightarrow \cdot F$
 $F \rightarrow \cdot (E)$
 $F \rightarrow \cdot \text{id}$

l_7

$T \rightarrow T * \cdot F$
 $F \rightarrow \cdot (E)$
 $F \rightarrow \cdot \text{id}$

l_8

$F \rightarrow (E \cdot)$
 $E \rightarrow E \cdot + T$

l_9

$E \rightarrow E + T \cdot$
 $T \rightarrow T \cdot * F$

l_{10}

$T \rightarrow T * F \cdot$

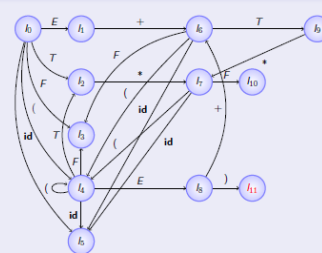
l_{11}

$F \rightarrow (E) \cdot$

tablica analizatora SLR

stan	id	działanie					przejście		
		+	*	(	)	\$	E	T	F
0	s5			s4			1	2	3
1		s6				akc			
2		r2	s7		r2	r2			
3		r4	r4		r4	r4			
4	s5			s4			8	2	3
5		r6	r6		r6	r6			
6	s5			s4				9	3
7	s5			s4					10
8		s6		s11					
9		r1	s7		r1	r1			
10		r3	r3		r3	r3			
11		r5	r5		r5	r5			

Diagram przejść stanów



I_{11}

5. $F \rightarrow (E) \cdot$

$nast(F)$

$*, +,), \$$

Algorytm 4.8. Konstruowanie tablicy analizatora SLR.

Wejście. Wzbogacona gramatyka G' .

Wyjście. Części akcja i przejście tablicy analizatora SLR dla gramatyki G' .

Metoda.

1. Zbuduj $C = \{I_0, I_1, \dots, I_n\}$, rodzinę zbiorów sytuacji LR(0) dla G' .
2. Stan i budujemy z I_i . Akcje analizatora dla stanu i wyznaczamy następująco:
 - a) jeśli $[A \rightarrow \alpha \cdot a\beta]$ jest w I_i oraz $przejście(I_i, a) = I_j$, to elementowi $akcja[i, a]$ nadajemy wartość „przesuń j ”; a musi być terminalem,
 - b) jeśli $[A \rightarrow \alpha \cdot]$ jest w I_i , to elementowi $akcja[i, a]$ nadajemy wartość „redukuj według $A \rightarrow \alpha$ ” dla wszystkich a z $FOLLOW(A)$; A nie może być równe S' ,
 - c) jeśli $[S' \rightarrow \cdot S]$ jest w I_i , to elementowi $akcja[i, \$]$ nadajemy wartość „akceptuj”.

Jeżeli wskutek stosowania powyższych reguł tworzymy sprzeczne akcje, mówimy, że gramatyka nie jest SLR(1). W takiej sytuacji algorytm nie generuje analizatora.

3. Wartości $przejście$ dla stanu i są tworzone dla wszystkich nieterminali A zgodnie z regułą: jeśli $przejście(I_i, A) = I_j$, to $przejście[i, A] = j$.
4. Wszystkie pozycje tablicy, którym w krokach 2. i 3. nie nadaliśmy wartości, oznaczamy jako błędne.
5. Stan startowy analizatora to stan zbudowany ze zbioru sytuacji, do którego należała sytuacja $[S' \rightarrow \cdot S]$. $\square$

Tablica analizatora, składająca się z wyznaczonych za pomocą algorytmu 4.8 funkcji akcji analizatora i przejść, jest nazywana *tablicą SLR(1) dla G* . Analizator LR używający tablicy SLR(1) dla G jest nazywany analizatorem SLR(1) dla G , a gramatyka, dla której istnieje tablica analizatora SLR(1), jest nazywana gramatyką SLR(1). Zazwyczaj opuszczamy (1) po SLR, gdyż nie będziemy się zajmować analizatorami, które podglądają więcej niż jeden symbol.

Ograniczenia analizy SLR - konflikty S/R i R/R

Rozpatrzmy bardzo uproszczoną gramatykę wyrażeń:

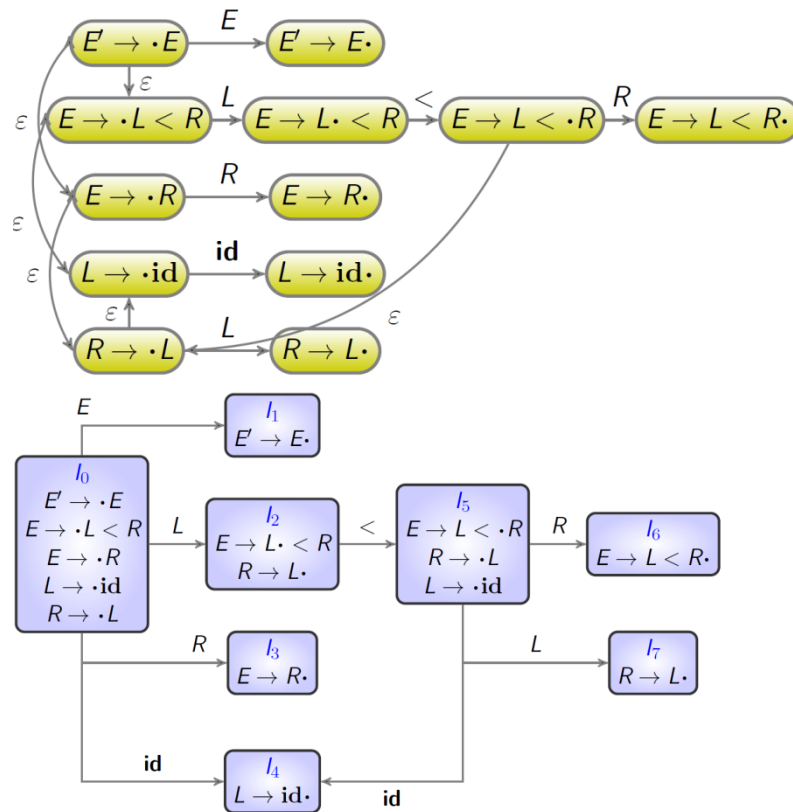
$E \rightarrow L < R$

$E \rightarrow R$

$L \rightarrow \text{id}$

$R \rightarrow L$

Gramatyka odpowiada wyrażeniom logicznym, gdzie porównywane są dwa wyrażenia arytmetyczne. Dla uproszczenia przyjęliśmy tylko jeden operator porównania. Wyrażenie może być też samym wyrażeniem arytmetycznym, którego też dla uproszczenia nie rozwijamy dalej, a tylko rozstawiliśmy z niego identyfikator. Jedyną cechą szczególną tej gramatyki jest to, że osobno rozwijamy lewy i prawy argument. Zbudujmy dla tej gramatyki automat.



Stan I2 zawiera sytuacje:

$E \rightarrow L \cdot < R$

$R \rightarrow L \cdot$

Co to oznacza? Do działania[2;<] możemy wstawić s4, ponieważ mamy sytuację $E \rightarrow L \cdot < R$. Jednocześnie druga sytuacja oznacza zwinienie reguły i widać, że < jest w nast(L), więc do tej samej komórki tablicy należałoby wstawić zwinienie. Tymczasem nie istnieje taki ciąg produkcji, który doprowadzi do uzyskania ciągu symboli $R <$, więc zwinienie w tym miejscu nie jest możliwe.

Rozwiązaniem może być uwzględnienie większego kontekstu w sytuacjach. (**kanoniczna analiza LR**)

Sytuacją LR(1) nazywamy sytuację LR(0) z dołączonym symbolem końcowym lub symbolem końca wejścia \$. Np. Dla reguły produkcji $A \rightarrow XY$ i symbolu końcowego a mamy:

$A \rightarrow \cdot XY, a$

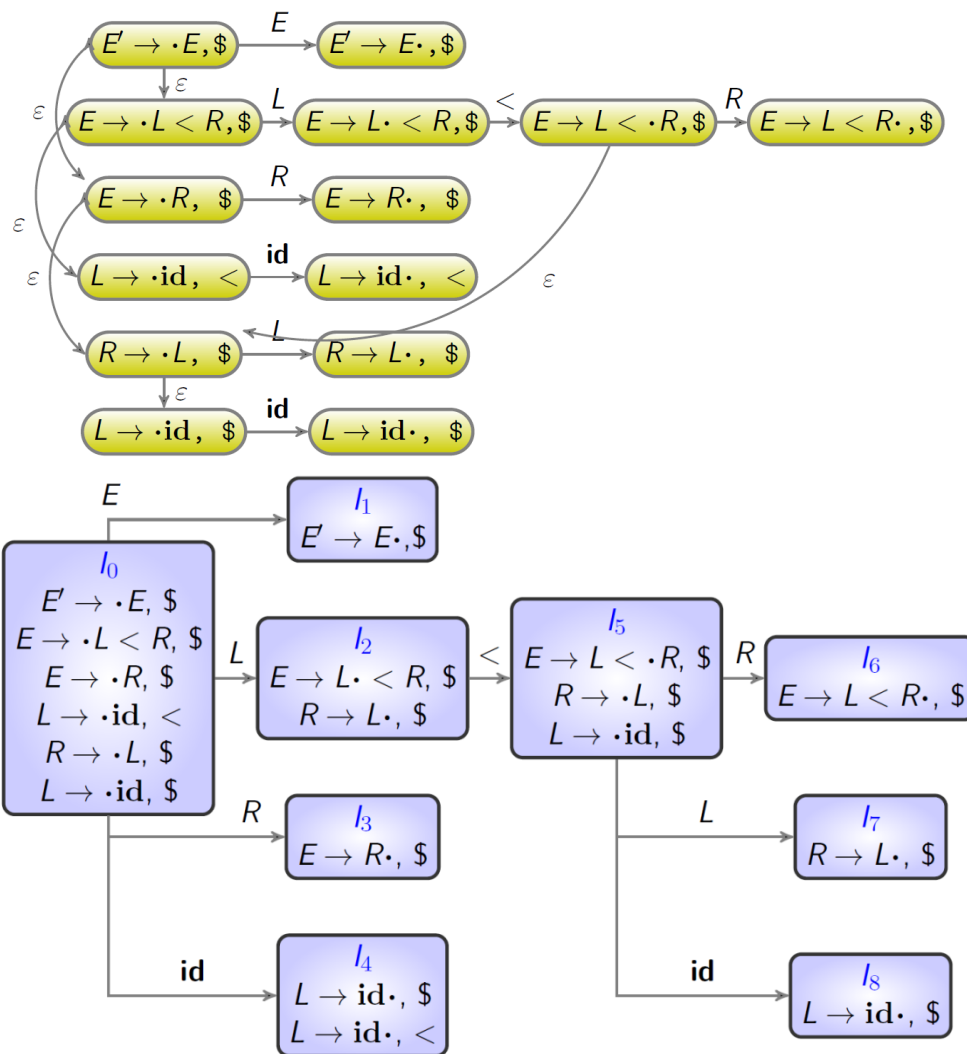
$A \rightarrow X \cdot Y, a$

$A \rightarrow XY \cdot, a$

podobnie dla symbolu końca wejścia \$.

Z takich sytuacji tworzony jest automat.

W kanonicznej analizie LR w stosunku do algorytmu dla SLR zmienia się nieznacznie zwinienie reguły.



Niestety analiza kanoniczna LR(1) produkuje bardzo duże tablice, nawet rząd wielkości większe niż w przypadku SLR(1). Rozmiar tablic można zmniejszyć. **Analizator LARL(1)** (Look-Ahead LR) ma dokładnie tyle samo stanów (tyle samo wierszy tablicy) co analizator SLR(1). Rozpoznaje większą klasę języków niż analizator SLR, ale nieznacznie mniejszą niż kanoniczny analizator LR. Konfliktów pomiędzy przesunięciem a zwinięciem (shift/reduce conflicts) unika się przez dokładną analizę kontekstu podglądanego symbolu. Tworzenie tablicy zaczyna się od utworzenia automatu SLR, czyli z użyciem sytuacji LR(0), które stanowią jądro analizatora LARL.

W starszych podręcznikach konstrukcji kompilatorów znaczną część tekstu zajmują analizatory korzystające z gramatyk operatorowych (metoda pierwszeństwa operatorów). Ich użyteczność jest bardzo ograniczona. Można za to wykorzystać pierwszeństwo (priorytet) operatorów do unikania konfliktów w analizatorach LR.

Tłumaczenie sterowane składnią

Dołączanie dalszych części kompilatora (analizy semantycznej, generowania i optymalizacji kodu) do analizy składniowej odbywa się zwykle na jeden z dwóch sposobów. Są to:

1. definicje sterowane składnią
2. schematy tłumaczenia

Pierwszy sposób jest bardziej ogólny. Drugi sposób wskazuje kolejność obliczania reguł semantycznych.

Definicje sterowane składnią

Każdej regule produkcji postaci $A \rightarrow \alpha$ przypisujemy zbiór reguł semantycznych postaci $b = f(c_1; c_2; \dots; c_k)$, gdzie f jest funkcją, zaś b i c_i mogą mieć jedno z dwóch znaczeń:

1. b jest atrybutem syntetyzowanym symbolem A , a $c_1; c_2; \dots; c_k$ są atrybutami symboli z prawej strony reguły produkcji
2. b jest atrybutem dziedziczonym jednego z symboli z prawej strony reguły produkcji, a $c_1; c_2; \dots; c_k$ są atrybutami symboli z reguły produkcji

W obu przypadkach atrybut b zależy od symboli $c_1; c_2; \dots; c_k$. Takie reguły zapisuje się jako fragmenty programu. Drzewo wyprowadzenia z atrybutami nazywa się **drzewem dekorowanym**, a sam proces obliczania wartości atrybutów procesem **dekorowania drzewa**.

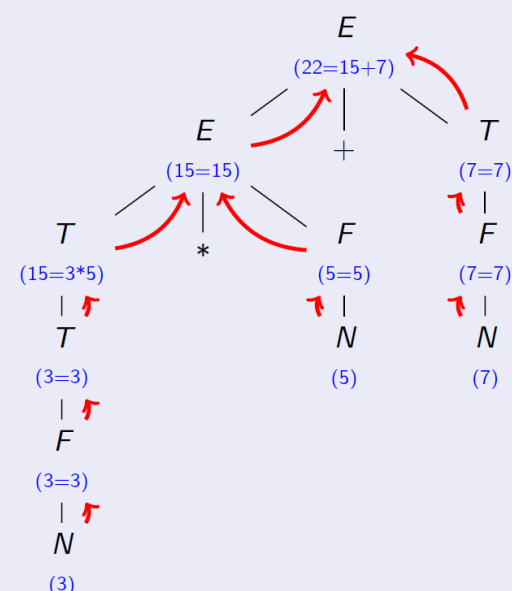
Atrybuty syntetyzowane

Definicję używającą tylko atrybutów syntetyzowanych nazywamy definicją **S-atrybutowaną**.

gramatyka (bison)

```
E: T { $$ = $1; }
    | E '+' T { $$ = $1 + $2; }
;
T: F { $$ = $1; }
    | T '*' F { $$ = $1 * $2; }
;
F: N { $$ = $1; }
;
```

drzewo wyprowadzenia

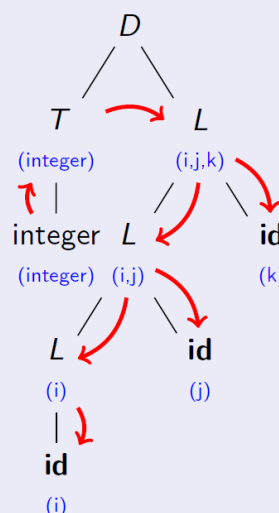


Atrybuty dziedziczone

gramatyka (bison)

```
D: T L { ins_list($2,$1); }
;
T: int { $$ = integer; }
    | real { $$ = real; }
;
L: id { $$ = list($1); }
    | L ',' id { $$ =
append($1,$2); }
;
```

drzewo wyprowadzenia



Analiza semantyczna – kontrola statyczna

Kompilator powinien sprawdzać reguły wykraczające poza gramatykę bezkontekstową. Na przykład:

1. Kontrola typów. Kompilator powinien zgłosić błąd, jeśli typy argumentów, parametrów itp. są niewłaściwe.
2. Kontrola przepływu sterowania. Kompilator powinien zgłosić błąd, jeśli instrukcja pojawi się w niewłaściwym kontekście, np. w przypadku braku instrukcji return w definicji funkcji zwracającej jakąś wartość.
3. Kontrola niepowtarzalności. Kompilator powinien zgłosić błąd, jeśli w tym samym zakresie widoczności deklarujemy zmienną o tej samej nazwie lub funkcję o tej samej nazwie i ew. sygnaturze (zależy od języka).
4. Kontrola dopasowania nazw. Np. w Adzie wiele konstrukcji wymaga podania tej samej nazwy na początku i na końcu.

Kontrola typów

Kontrola typów może być realizowana statycznie poprzez sprawdzanie równoważności typów. Jest to procedura rekurencyjna, gdyż typy mogą być definiowane za pomocą innych typów, a takie konstrukcje mogą być dodatkowo nazywane. Niektóre typy dodatkowo mogą być uznawane za zgodne, przy czym zgodność może występować tylko w jednym kierunku.

Możliwa i stosowana jest też dynamiczna kontrola typów, której może towarzyszyć przekształcanie typów. Np. wartości całkowite mogą być w wyrażeniach zamieniane na wartości zmiennoprzecinkowe, jeśli w tym samym wyrażeniu występują wartości zmiennoprzecinkowe.

Podział pamięci

Kompilator powinien odpowiednio podzielić pamięć, którą program otrzyma od systemu operacyjnego w czasie wykonania.

kod
dane statyczne
stos
sterta

Rozmiar kodu jest znany w czasie kompilacji. Pamięć dla kodu może być przydzielona w pamięci, której zawartości nie można zmieniać.

Rozmiar danych statycznych jest także znany w czasie kompilacji.

Rozmiar stosu zmienia się w trakcie wykonywania programu. Na stosie umieszczane są rekordy aktywacji podprogramów i bloków, w tym dane tymczasowe. Stos rośnie w dół.

Rozmiar sterty zmienia się w trakcie wykonywania programu. Sterta zawiera obiekty, dla których pamięć przydzielana jest dynamicznie. Dla niektórych języków programowania sterta może przechowywać rekordy aktywacji. Sterta rośnie w górę.

Strategie rezerwacji pamięci

Dla różnych danych mogą być stosowane różne strategie rezerwacji pamięci.

1. Przydział statyczny stosowany jest do wszystkich obiektów (w tym kodu programu) znanych w chwili kompilacji programu.
2. Przydział stosowy stosowany jest do danych lokalnych i tymczasowych, których czas życia ograniczony jest do czasu życia podprogramu lub bloku w którym występują. Po zakończeniu podprogramu/bloku pamięć jest zwalniana, a jej zawartość może zostać nadpisana przez inne dane.
3. Przydział stertowy jest stosowany do obiektów, których rozmiar nie jest znany w chwili kompilacji programu lub których czas życia nie pokrywa się z czasem życia podprogramów. Istnieją różne algorytmy zarządzania stertą.

Rekordy aktywacji

Wykonanie podprogramu (procedury lub funkcji) nie polega wyłącznie na skoku do innego fragmentu kodu z zapamiętaniem adresu powrotu. Sekwencje wywołania podprogramu i powrotu z podprogramu muszą zadbać o wiele zagadnień:

1. przekazywanie parametrów
2. zwracanie wartości przez funkcje
3. przydział pamięci i określanie adresów zmiennych lokalnych
4. określanie adresów zmiennych nielokalnych
5. zapamiętanie i przywrócenie stanu procesora

Podział tych zadań pomiędzy kod wywołujący a wołany podprogram może być różny. Zależy od architektury komputera. Mniejszy kod uzyskuje się na ogół przez umieszczenie większej części zadań w podprogramie wywoływanym.

zwracana wartość
parametry aktualne
wiązanie sterowania
wiązanie dostępu
stan procesora
dane lokalne
dane tymczasowe

Zwracana wartość może być przekazywana przez rejestr, co jest szybsze.

Przy parametrach aktualnych przynajmniej niektóre typy wartości mogą być także wydajniej przekazywane w rejestrach.

Wiązanie sterowania wskazuje rekord aktywacji procedury wywołującej.

Wiązanie dostępu jest używane do odwoływania się do danych nielokalnych umieszczonych w rekordach aktywacji innych podprogramów. W niektórych językach, np. w Fortranie, nie jest potrzebne. Może być zastąpione przez tablice Display.

Stan procesora obejmuje zawartość rejestrów procesora, w tym licznika rozkazów i wskaźnika stosu. Dane te powinny być przywrócone po powrocie z podprogramu.

Dane lokalne zawierają dane deklarowane wewnątrz podprogramu. Dane mogą być przechowywane z uwzględnieniem wyrównania do pełnych słów itp.

Stos może być wykorzystywany do przechowywania tymczasowych wyników obliczeń, np. przy zastosowaniu odwrotnej notacji polskiej.

Dostęp do danych nielokalnych w przypadku podprogramów zagnieżdżonych realizuje się przez wiązanie dostępu. Wskazuje ono rekord aktywacji ostatniego wywołania podprogramu otaczającego dany podprogram. Dostęp do podprogramu na wyższym poziomie można uzyskać przechodząc w górę po wiązaniach dostępu liczbą razy równą różnicy poziomów statycznego zagłębienia. Zamiast chodzenia po wiązaniach dostępu można korzystać z tablic Display. Tablice te zawierają wskaźniki do rekordów aktywacji ostatnich wywołań kolejnych otaczających podprogramów.